

SC3010 Computer Security
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28

Contents

1	Foundations: CIA Triad, Threats, Risk and Attack Surfaces	1
1.1	What Are We Protecting? The CIA Triad	1
1.2	Threats, Vulnerabilities, Attacks	2
1.3	Risk: Likelihood $\times$ Impact	3
1.4	Attack Surfaces	4
1.5	Design Principles: Least Privilege and Defence in Depth	5
1.6	Chapter Notes	5
1.7	Exercises	5
2	Cryptographic Tools: AES, RSA, Hashes and PKI	7
2.1	Symmetric Encryption and AES	7
2.2	Asymmetric Encryption and RSA	8
2.3	Cryptographic Hashes	9
2.4	Hybrid Encryption and Key Exchange	10
2.5	Public-Key Infrastructure (PKI)	11
2.6	Chapter Notes	11
2.7	Exercises	12
3	Authentication: Passwords, MFA and Protocols	13
3.1	Passwords: Intuition to Formal Storage	13
3.2	MFA and Biometrics	14
3.3	Challenge–Response, Needham–Schroeder, Kerberos	15
3.4	Exercises	17

4	Access Control: ACLs, RBAC, ABAC and Lattices	18
4.1	The Access Matrix: Columns Are ACLs, Rows Are Capabilities	18
4.2	RBAC: Roles as Indirection	19
4.3	ABAC: Policies over Attributes	20
4.4	Lattice Model: Classifying Information Flow	21
4.5	Exercises	22
5	Software Security: Buffer Overflows, Injection, and Hardening	23
5.1	Why Software Fails: Trusting Data as Control	23
5.2	Buffer Overflows: Overwriting the Stack	24
5.3	Injection I: SQL Injection	25
5.4	Injection II: Cross-Site Scripting (XSS)	26
5.5	Hardening: Canaries, ASLR, DEP, and Input Validation	26
5.6	Exercises	27
6	Network Security: Firewalls, IDS/IPS, TLS and VPNs	29
6.1	The Network Boundary	29
6.2	Firewalls: Packet Filter, Stateful, Application	30
6.3	IDS and IPS: Signature vs. Anomaly	31
6.4	TLS 1.3 Handshake: Whispering in Public	32
6.5	VPNs: Private Tunnels over Public Roads	32
6.6	Exercises	33
7	Operating System Security: Isolation and Containment	35
7.1	Why the OS Must Isolate	35
7.2	The Reference Monitor and Complete Mediation	36
7.3	Privilege Separation	37
7.4	Sandboxing	38
7.5	Virtualization and Container Security	38
7.6	Putting It Together: Hardening Checklist	39
7.7	Exercises	40

8 Applications: Privacy, Forensics, Incident Response, and Law	41
8.1 Privacy: From Removing Names to Formal Guarantees	41
8.2 Differential Privacy	42
8.3 Digital Forensics and Chain-of-Custody	43
8.4 Incident Response: The NIST Lifecycle	44
8.5 Regulation: GDPR Essentials	45
8.6 Budget, Custody, and Notification: A Worked Scenario	45
8.7 Exercises	46

Chapter 1

Foundations: CIA Triad, Threats, Risk and Attack Surfaces

“Security is a process, not a product.” — Bruce Schneier. Before we encrypt anything we must agree on what we are protecting, from whom, at what cost, and where the doors are.

From zero: no prior security knowledge assumed. We build from everyday intuition—a locked diary, a tampered letter, a shop that must stay open—to formal definitions of confidentiality, integrity and availability. Every term is defined on first use. This chapter is the vocabulary for the whole book.

Learning Outcomes

After this chapter you will be able to:

- (L1) state the CIA triad and classify a security requirement under C, I or A;
- (L2) distinguish threat, vulnerability, attack and exploit with examples;
- (L3) apply STRIDE to enumerate threats for a simple system;
- (L4) compute and prioritise risk as $\text{likelihood} \times \text{impact}$ (qualitative and quantitative);
- (L5) identify and reduce the attack surface of a system across network, software, human and physical layers.

1.1 What Are We Protecting? The CIA Triad

Intuition first. A diary has three wishes: nobody else reads it (secrecy), nobody rewrites a page without you noticing (accuracy), and you can open it when you want (reachability). Computer security generalises these to *confidentiality*, *integrity* and *availability*—the CIA triad. Almost every requirement maps to one or more of them.

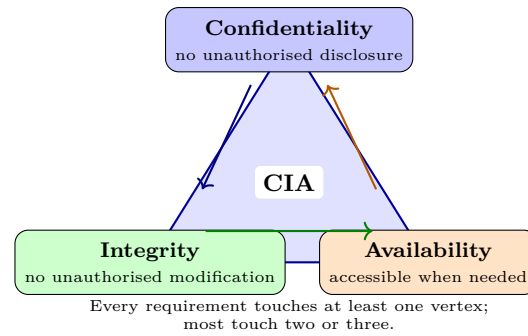


Figure 1.1: The CIA triad. Confidentiality protects disclosure, integrity protects modification, availability protects access. Tension between them drives design trade-offs.

Definition 1.1 (CIA triad). • **Confidentiality:** information is not disclosed to unauthorised entities. Enforced by access control, encryption, least privilege.

- **Integrity:** information and systems are not altered in an unauthorised or undetectable way. Covers data integrity and origin authenticity. Enforced by hashes, MACs, signatures.
- **Availability:** authorised users can access information and services when needed. Enforced by redundancy, rate-limiting, DoS mitigation.

Overlap is normal. A signed software update needs integrity (not tampered) *and* authenticity (from the vendor, hence confidentiality of the signing key) *and* availability (patch must arrive). Two later refinements appear often: *authenticity* and *non-repudiation* are usually treated as facets of integrity; *accountability* spans all three via logging.

Example 1.1 (Classifying requirements). Online exam: (a) student A cannot see student B’s answers — confidentiality; (b) submitted answers cannot be altered after deadline — integrity; (c) submission portal stays up during the window — availability. Encrypting the database helps (a) but does not help (c).

Trade-offs matter. Requiring a 20-character password and hardware token improves confidentiality but can hurt availability (users locked out) and even integrity (users write passwords on sticky notes). Good security balances the triangle for the asset at hand.

1.2 Threats, Vulnerabilities, Attacks

Picture a house: a burglar who wants jewellery is the *threat actor*, an unlocked window is the *vulnerability*, climbing in is the *attack*, the crowbar used is the *exploit*, and the jewellery is the *asset*. Security language follows the same chain.

Definition 1.2 (Core terms). An **asset** is what we value (data, service, reputation). A **vulnerability** is a weakness that can be exploited (bug, misconfiguration, human habit). A **threat** is a potential event that could harm an asset via a vulnerability. A **threat actor** is who or what carries it out (insider, criminal, nation-state, botnet). An **attack** is the realised threat; an **exploit** is the concrete technique or code.

Enumerating threats systematically helps. **STRIDE** (Microsoft) asks six questions per component: **S**poofing (authenticity), **T**ampering (integrity), **R**epudiation (non-repudiation), **I**nformation disclosure (confidentiality),

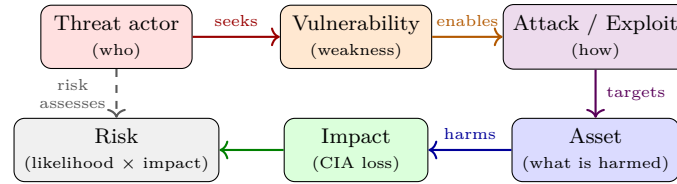


Figure 1.2: From actor to risk: a threat actor exploits a vulnerability to attack an asset, causing an impact measured as risk.

Denial of service (availability), Elevation of privilege (authorisation). Walk each data flow and store through STRIDE and you rarely miss a class.

Example 1.2 (STRIDE on a file upload). Upload endpoint: spoofing — attacker impersonates a user via stolen cookie; tampering — modifies file in transit; repudiation — denies uploading malware; information disclosure — reads other users’ files via path traversal; DoS — uploads huge files to fill disk; elevation — uploads a web shell to gain server execution. One feature, six lenses.

We also distinguish *attack vectors* (path in: network, USB, phishing email) from *attack surfaces* (next section).

Remark 1.1 (Threat modelling steps). A lightweight workflow: (1) draw the data-flow diagram, (2) walk each element through STRIDE, (3) score each threat by risk, (4) decide mitigate/transfer/accept/avoid, (5) record residual risk and review after changes. Tools like OWASP Threat Dragon automate (1)–(3) but judgement in (4) remains human.

Example 1.3 (STRIDE on a login form). Login: spoofing — credential stuffing with leaked passwords; tampering — parameter tampering on **next** URL to phish; repudiation — attacker denies brute-force attempts without logs; information disclosure — verbose “user exists” errors enumerate accounts; DoS — rate-limit bypass locks accounts; elevation — SQL injection in username field. Mitigations map directly: MFA, input validation, audit logging, generic errors, rate limiting, prepared statements.

1.3 Risk: Likelihood $\times$ Impact

Not every threat deserves the same budget. *Risk* lets us prioritise. Intuitively: how likely is it, and how bad if it happens?

Definition 1.3 (Risk). **Risk** is the expected loss from a threat exploiting a vulnerability. Qualitatively,

$$\text{Risk} = \text{Likelihood} \times \text{Impact}, \quad (1.1)$$

often scored on a matrix (e.g., 3×3 or 5×5). Quantitatively, annualised loss expectancy $\text{ALE} = \text{ARO} \times \text{SLE}$ where ARO is annualised rate of occurrence and SLE single loss expectancy.

Example 1.4 (ALE). A phishing breach costs $\text{SLE} = \$80,000$ and occurs $\text{ARO} = 0.3/\text{year}$ without training. $\text{ALE} = \$24,000/\text{year}$. Training at $\$8,000/\text{year}$ that halves ARO to 0.15 saves $\$12,000$ in ALE — net benefit $\$4,000$.

Listing 1.1: Risk register ranking (qualitative 5×5).

```
1 def risk_score(likelihood, impact):
```

```

2      # both 1..5  -> 1..25
3      assert 1 <= likelihood <= 5 and 1 <= impact <= 5
4      return likelihood * impact
5
6      risks = [("SQLi on login", 4, 5), ("USB drop in lobby", 2, 4), ("CDN outage", 3, 3)]
7      for name, L, I in sorted(risks, key=lambda r: risk_score(r[1], r[2]), reverse=True):
8          print(name, risk_score(L, I))
9      # SQLi=20 (critical), USB=8 (medium), CDN=9 (medium)

```

Risk handling has four options: *mitigate* (patch, add control), *transfer* (insurance, outsource), *accept* (low risk, cost exceeds benefit), *avoid* (remove the feature). Residual risk — what remains after controls — is what management signs off on. Threat modelling without risk ranking is just a wish list.

1.4 Attack Surfaces

The *attack surface* is the sum of all points where an attacker can interact with the system. Smaller surface, fewer places to be wrong.

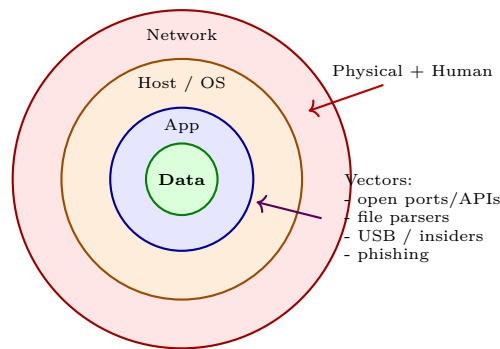


Figure 1.3: Layered attack surface: network $\supset$ host $\supset$ application $\supset$ data, with human and physical vectors cutting across. Reducing surface means fewer exposed interfaces at each ring.

Dimensions to audit: **network** (open ports, APIs, protocols, wireless), **software** (input parsers, dependencies, plugins, debug endpoints), **human** (phishing susceptibility, insider actions, shadow IT), **physical** (USB ports, unlocked terminals, discarded drives). A classic reduction playbook: close unused ports, disable unused features and accounts, enforce least privilege, validate every input at the trust boundary, keep dependencies minimal and patched.

Example 1.5 (Surface reduction). A campus printer exposing Telnet, FTP, a web panel and mDNS has a network surface of four services. Disabling Telnet/FTP (unused), putting the panel behind VPN and SSO, and turning off mDNS outside the subnet cuts the reachable surface by three quarters without buying anything.

Attack surface analysis pairs with *trust boundaries*: draw where data crosses from untrusted to trusted (browser $\rightarrow$ server, server $\rightarrow$ DB). Every crossing needs validation, authentication and logging.

1.5 Design Principles: Least Privilege and Defence in Depth

Two principles guide every later chapter. **Least privilege** says each component gets only the permissions it needs and no more, for the shortest time needed. A PDF viewer that can launch PowerShell violates it; a web app running as root does too. Enforce via roles, capabilities, sandboxing and short-lived tokens.

Defence in depth (layered security) says no single control should be a single point of failure. An attacker who phishes a password still faces MFA; who bypasses MFA still faces network segmentation; who reaches the DB still faces encrypted rows and audit logs. Each layer assumes the outer one has already fallen.

Other Saltzer–Schroeder principles reinforce these: *fail-safe defaults* (deny by default), *complete mediation* (check every access), *open design* (security must not depend on secrecy of the design), *separation of duties* (no one person can both request and approve a wire transfer), *least common mechanism* and *psychological acceptability* (controls users can actually use).

Definition 1.4 (Threat model). A **threat model** is a structured description of what you defend, whom you defend against, and what you assume the attacker can and cannot do. It fixes scope: network attacker vs. local attacker, honest-but-curious vs. fully malicious, computational bounds, physical access.

Example 1.6 (Threat model matters). Full-disk encryption protects against a stolen laptop (attacker gets disk, not key) but not against malware running while you are logged in (attacker is inside the trust boundary). Same tool, different model, different outcome. State the model before claiming “secure.”

Risk, surface and principles together drive the security lifecycle: *identify* assets and threats → *assess* risk → *mitigate* via layered, least-privilege controls that shrink the surface → *verify* and *monitor*.

Remark 1.2 (Common pitfall). “We encrypt, therefore we are secure” ignores the model. Encryption without integrity invites padding-oracle attacks; availability without rate limits invites DoS; least privilege without audit invites insider misuse. Always ask *which* CIA property, against *which* threat model, at *what* residual risk.

1.6 Chapter Notes

Further reading: Anderson’s *Security Engineering* (3rd ed.) for CIA trade-offs in real systems; NIST SP 800-30 for risk assessment and 800-154 for threat modelling. STRIDE is documented in Shostack’s *Threat Modeling: Designing for Security*. For attack surface metrics see Manadhata and Wing (2011). Common Criteria (ISO 15408) formalises the asset–threat–control vocabulary used here. Saltzer and Schroeder (1975) remains the classic on design principles—read Section 2 before any system design review.

1.7 Exercises

E1.1 **CIA classification.** For each, name the primary CIA property (and any secondary): (a) ransomware encrypts a hospital’s records; (b) an attacker replays a captured payment request; (c) a CDN caches a private profile page; (d) a sensor sends stale temperature readings due to a dead battery. Justify in one sentence each.

- E1.2 **Threat vs. vulnerability vs. attack.** Give concrete examples (one sentence each) showing they are distinct: a vulnerability with no known threat, a threat with no current vulnerability, and an attack that failed because the vulnerability had been patched. Explain why conflating them misallocates budget.
- E1.3 **STRIDE walk.** Draw a data-flow diagram for a password-reset via email (browser $\rightarrow$ web app $\rightarrow$ mail server $\rightarrow$ browser). List one STRIDE threat per element/flow and state the CIA impact. Which two would you fix first and why?
- E1.4 **Risk arithmetic.** Two risks: A has likelihood $4/5$ and impact \$200,000; B has likelihood $2/5$ and impact \$500,000. Compute expected loss for each. If a \$30,000 control halves B's likelihood, is it worthwhile? How does a 5×5 qualitative matrix ($L, I \in \{1..5\}$) change the ranking versus expected value?
- E1.5 **Attack surface audit.** Pick a real system you use (e.g., your laptop, a lab router, or the NTU Learn app). Enumerate at least six attack-surface elements across network/software/human/physical, mark the trust boundaries, and propose three concrete reductions. Estimate before/after surface qualitatively.

Chapter 2

Cryptographic Tools: AES, RSA, Hashes and PKI

“Cryptography is not magic—it is mathematics plus careful engineering.” We turn intuition (locked boxes, fingerprints) into precise tools: symmetric ciphers for speed, asymmetric ciphers for key distribution, hashes for integrity, and PKI for trust at scale.

From zero: high-school arithmetic to real crypto. We assume no prior cryptography. Start from “what does it mean to hide a message?” then formalise encryption, see why one key is not enough, build RSA from modular arithmetic, and end with how your browser trusts a server it has never met.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain symmetric encryption and AES structure at a conceptual level;
- (L2) describe RSA key generation, encryption and decryption and why it works;
- (L3) state the three hash properties and choose the right one for a task;
- (L4) construct a certificate chain and verify it to a trust anchor;
- (L5) select symmetric vs. asymmetric vs. hash vs. PKI for a given requirement.

2.1 Symmetric Encryption and AES

Intuition: Alice and Bob share one physical key that both locks and unlocks the same box. Anyone with the key can do both. That is *symmetric* encryption: one secret key k for both directions. Formally a scheme is $(\text{Gen}, \text{Enc}, \text{Dec})$ with $\text{Dec}_k(\text{Enc}_k(m)) = m$ and ciphertext reveals nothing about m without k .

The one-time pad (Vernam) is perfect: $c = m \oplus k$, $|k| = |m|$, random k used once. Shannon proved it has

perfect secrecy, but the key must be as long as the message and never reused—impractical. Modern ciphers trade perfection for efficiency with a short reusable key and computational security.

AES (Advanced Encryption Standard) is the standard symmetric cipher: block size 128 bits, key 128/192/256 bits, 10/12/14 rounds. Each round is a substitution–permutation network:

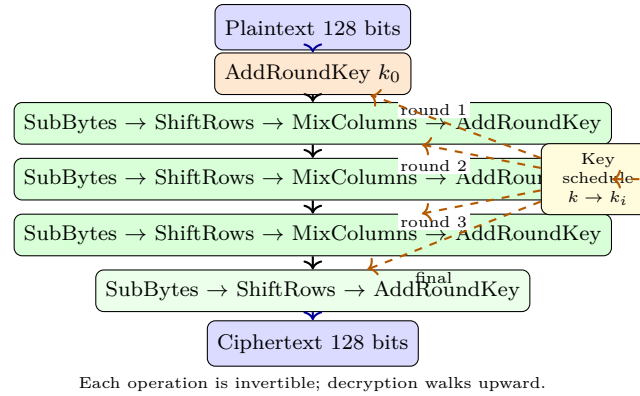


Figure 2.1: AES round structure (128-bit key, 10 rounds shown schematically). SubBytes adds non-linearity, ShiftRows and MixColumns diffuse bits, AddRoundKey mixes the key. Repetition gives confusion and diffusion.

Confusion (SubBytes) hides the relation between key and ciphertext; diffusion (ShiftRows + MixColumns) spreads one plaintext bit over many ciphertext bits. Repeating 10–14 times makes the cipher look random without the key.

AES encrypts blocks; long messages need a *mode*. ECB (each block independently) leaks patterns and must be avoided. Use **CBC** or **CTR** with a random IV, or better **GCM** (CTR plus authentication). GCM provides *authenticated encryption*: confidentiality and integrity together. Never invent your own mode.

The hard part is not AES itself but *key distribution*: how do Alice and Bob share k without meeting? Sending k in the clear defeats the purpose. We need a second tool.

Listing 2.1: AES-GCM encrypt/decrypt (conceptual; use a real library).

```

1 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
2 import os
3 key = AESGCM.generate_key(bit_length=128)    # 16 bytes, keep secret
4 aesgcm = AESGCM(key)
5 nonce = os.urandom(12)                       # 96-bit, never reuse with same key
6 ct = aesgcm.encrypt(nonce, b"hello SC3010", associated_data=None)
7 pt = aesgcm.decrypt(nonce, ct, associated_data=None) # raises if tampered
8 assert pt == b"hello SC3010"
  
```

2.2 Asymmetric Encryption and RSA

Intuition: a mailbox with an open slot (public key) anyone can drop letters into, but only the owner with the private key can open. *Asymmetric* (public-key) encryption uses a key pair (pk, sk) : pk encrypts, sk decrypts. No prior shared secret needed.

RSA (Rivest–Shamir–Adleman, 1977) rests on modular arithmetic. Recall $a \equiv b \pmod{n}$ means $n \mid (a - b)$, and $\gcd(a, n) = 1$ implies a has an inverse modulo n .

Definition 2.1 (RSA). Generate two distinct large primes p, q , set $n = pq$, $\phi(n) = (p-1)(q-1)$. Choose e with $\gcd(e, \phi(n)) = 1$, compute $d \equiv e^{-1} \pmod{\phi(n)}$. Public key $pk = (n, e)$, private key $sk = (n, d)$. Encryption of $m \in [0, n)$:

$$c \equiv m^e \pmod{n}, \quad (2.1)$$

decryption:

$$m \equiv c^d \pmod{n}. \quad (2.2)$$

Why does it work? Euler's theorem: if $\gcd(m, n) = 1$, $m^{\phi(n)} \equiv 1 \pmod{n}$. Since $ed \equiv 1 \pmod{\phi(n)}$, write $ed = 1 + k\phi(n)$. Then

$$c^d \equiv (m^e)^d = m^{ed} = m^{1+k\phi(n)} = m \cdot (m^{\phi(n)})^k \equiv m \cdot 1^k \equiv m \pmod{n}. \quad (2.3)$$

With Chinese remainder handling for $\gcd(m, n) \neq 1$ the same holds. Security rests on hardness of *factoring* n to recover $\phi(n)$ and d ; with 2048-bit n , factoring is infeasible today.

Theorem 2.1 (RSA correctness). For every $m \in \mathbb{Z}_n$, $\text{Dec}_{sk}(\text{Enc}_{pk}(m)) = m$ when keys are generated as above.

RSA is slow (modular exponentiation of huge numbers) and malleable without padding. Never use textbook RSA. Use **OAEP** padding for encryption and **PSS** for signatures, or better use hybrid encryption: RSA (or Diffie–Hellman) to transport a random AES key, then AES for bulk data.

Listing 2.2: RSA toy (small primes) — do not use for real secrecy.

```

1  import math
2  p, q = 61, 53
3  n = p * q
4  phi = (p-1)*(q-1)
5  e = 17 # coprime to phi
6  d = pow(e, -1, phi) # Python 3.8+: modular inverse
7  def enc(m): return pow(m, e, n)
8  def dec(c): return pow(c, d, n)
9  assert dec(enc(42)) == 42
10 print(f"n={n} phi={phi} e={e} d={d}") # n=3233 d=2753

```

Key sizes: RSA-2048 $\approx$ 112-bit symmetric security; RSA-3072 $\approx$ 128-bit. For new systems prefer elliptic-curve (ECDH/ECDSA) or post-quantum KEMs where available—same ideas, smaller keys.

2.3 Cryptographic Hashes

A hash is a fingerprint: arbitrary input $\rightarrow$ fixed output, easy to compute, hard to invert. Formally $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ with $n = 256$ for SHA-256.

Definition 2.2 (Hash security). H is (a) **preimage resistant** if given y it is hard to find x with $H(x) = y$; (b) **second-preimage resistant** if given x it is hard to find $x' \neq x$ with $H(x') = H(x)$; (c) **collision resistant** if hard to find *any* $x \neq x'$ with $H(x) = H(x')$. Collision resistance implies second-preimage resistance.

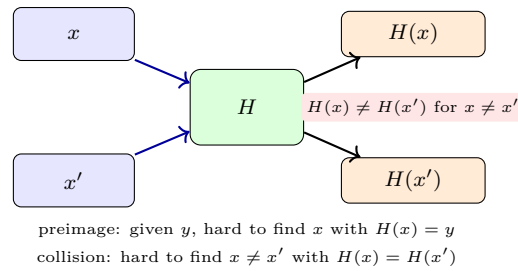


Figure 2.2: Hash properties visualised. Even one-bit change avalanches the digest; finding any preimage or collision should be infeasible (2^{256} work for SHA-256).

Applications: file integrity ($H(\text{file})$ detects tampering), password storage (store salted slow hash, not plaintext), commitments, Merkle trees for blockchains. For integrity *with a key*, use **HMAC**: $\text{HMAC}_k(m) = H((k \oplus \text{opad}) \| H((k \oplus \text{ipad}) \| m))$, or an AEAD mode directly.

Birthday bound: collisions for n -bit hash need $\approx 2^{n/2}$ work by the birthday paradox, so SHA-256 gives 128-bit collision security. MD5 and SHA-1 are broken for collisions—do not use.

Listing 2.3: SHA-256 and HMAC.

```

1 import hashlib, hmac
2 msg = b"SC3010 is fun"
3 print(hashlib.sha256(msg).hexdigest())
4 # hmac with a secret key
5 key = b"secret-key-16bytes"
6 tag = hmac.new(key, msg, hashlib.sha256).hexdigest()
7 # verify by recomputing and comparing with hmac.compare_digest
8 assert hmac.compare_digest(tag, hmac.new(key, msg, hashlib.sha256).hexdigest())

```

2.4 Hybrid Encryption and Key Exchange

In practice we never choose one tool alone. **Hybrid encryption** uses asymmetric crypto to transport a fresh symmetric key, then symmetric crypto for bulk data—combining key-distribution without a shared secret with AES speed.

Bulk via AES, key via RSA/ECDH. Sender picks random $k \leftarrow \{0, 1\}^{128}$, computes $c_1 = \text{Enc}_{pk}(k)$ with RSA-OAEP or ECIES, $c_2 = \text{Enc}_k^{\text{AES-GCM}}(m)$, sends (c_1, c_2) . Receiver recovers $k = \text{Dec}_{sk}(c_1)$ then m . Even if m is gigabytes, only 256 bytes use slow public-key work.

Diffie–Hellman (DH) offers an alternative without encryption: Alice picks a , sends $g^a \bmod p$; Bob picks b , sends $g^b \bmod p$; both compute $g^{ab} \bmod p$ as the shared secret. An eavesdropper sees g^a, g^b but not g^{ab} (discrete-log hardness). DH needs authentication—otherwise a man-in-the-middle substitutes both sides—so it is paired with signatures or PKI (next section). Modern TLS 1.3 uses ephemeral ECDH (curve P-256 or X25519) for forward secrecy: even if the long-term key leaks later, past g^{ab} values remain secret because a, b were ephemeral.

Definition 2.3 (Forward secrecy). A key-exchange has **forward secrecy** if compromise of long-term keys does not reveal past session keys. Ephemeral DH/ECDH provides it; static RSA key transport does not.

Example 2.1 (Why hybrid matters). Encrypting a 10 MB video with RSA-2048 directly would need $> 40,000$

RSA blocks and minutes of CPU. Hybrid does one RSA operation (milliseconds) plus 10 MB of AES-GCM at gigabytes per second. Use the right tool for each job.

2.5 Public-Key Infrastructure (PKI)

Asymmetric crypto solves key distribution but creates a new question: whose public key is this? Without binding, an attacker substitutes their own pk . PKI binds names to keys via certificates.

A **certificate** is $\text{Cert} = \langle \text{subject}, pk_{\text{subject}}, \text{validity}, \text{issuer}, \text{extensions}, \sigma \rangle$ where $\sigma = \text{Sign}_{sk_{\text{issuer}}}(H(\text{rest}))$. A **Certificate Authority (CA)** is an entity trusted to issue and revoke certificates. Your device ships with a *trust store* of root CA public keys (trust anchors).

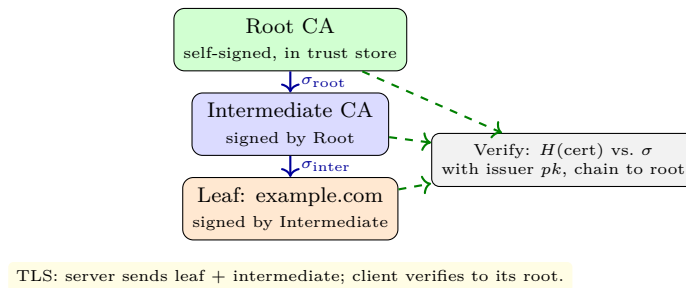


Figure 2.3: Certificate chain of trust. Each certificate is signed by its issuer; verification walks upward until a trust anchor. Revocation via CRL/OCSP prunes broken links.

Verification: given leaf for **example.com**, fetch intermediate, verify σ_{inter} on leaf with pk_{inter} , then verify σ_{root} on intermediate with pk_{root} from trust store, check validity periods, name matches, and revocation (CRL or OCSP). If all pass, pk_{leaf} is trusted for that name. TLS (HTTPS) does exactly this during the handshake before any AES session key is negotiated.

Threats to PKI: CA compromise, mis-issuance, name-constraint bypass, revoked-but-still-trusted certs. Mitigations include Certificate Transparency (public logs of all certs), short-lived certs, OCSP stapling, and pinning.

Example 2.2 (What PKI does not solve). PKI authenticates the *server name*, not that the server is honest. A valid certificate for **evil-phishing.example** passes PKI but the site still steals passwords. Users must also check the name matches their intent—hence EV deprecation and browser warnings for look-alike domains (homograph attacks). PKI is necessary but not sufficient; user education and allow-lists matter too.

Remark 2.1 (Key management in practice). Generate keys in an HSM or OS keystore, never hard-code them; rotate regularly; back up the private key securely or accept that loss means re-issuance; use ACME (Let’s Encrypt) to automate short-lived certificates and avoid expiry outages that still top outage post-mortems every year.

2.6 Chapter Notes

For AES internals see Daemen and Rijmen, *The Design of Rijndael*; for modes and AEAD see NIST SP 800-38A/D. RSA is from Rivest, Shamir, Adleman (1978); textbook vs. padded RSA is covered in Katz and Lindell, *Introduction to Modern Cryptography*. Hash functions: NIST FIPS 180-4 (SHA-2) and 202 (SHA-3). PKI and

TLS: RFC 5280 (certificates) and RFC 8446 (TLS 1.3); Shostack and Gutmann give practitioner views. For post-quantum migration see NIST PQC standards (2024).

2.7 Exercises

- E2.1 AES and modes.** Why does AES-ECB leak plaintext patterns? Sketch a 2×2 pixel image where ECB preserves the outline. Which mode(s) would you recommend for (a) disk encryption with random access and (b) a TLS record layer, and why?
- E2.2 RSA by hand.** Let $p = 17$, $q = 23$, $e = 5$. Compute n , $\phi(n)$, d , then encrypt $m = 42$ and decrypt. Show Euler step $ed = 1 + k\phi(n)$ explicitly. What goes wrong if you reuse n with two different e values across users?
- E2.3 Hash properties.** Order the three hash properties by strength (which implies which). Give an application that needs only second-preimage resistance but not collision resistance, and one that needs collision resistance. Explain why MD5 is unsafe for signatures even if preimage still seems hard.
- E2.4 HMAC vs. plain hash.** Why is $\text{HMAC}_k(m)$ preferred over $H(k\|m)$ for message authentication? Describe a length-extension attack on SHA-256 and how HMAC defeats it. When would you use HMAC versus AES-GCM?
- E2.5 PKI verification.** A browser receives leaf for `shop.example` signed by Inter-CA, which is signed by Root R in its trust store. List the five checks the browser performs before showing a padlock. An attacker compromises Inter-CA but not Root—what can they forge, and how do CT logs and OCSP help detect and contain it?

Chapter 3

Authentication: Passwords, MFA and Protocols

“Authentication is proving you are who you claim to be.” — To a computer you are just bits on a wire; we must bind those bits to a human using something you know, have, or are.

From zero: no crypto or security background assumed. We start from everyday logins, build intuition with pictures, then formalise passwords, entropy, hashing, salting, multi-factor and biometrics, and finally dissect real protocols—challenge–response, Needham–Schroeder, and Kerberos. Every term is defined on first use.

Learning Outcomes

After this chapter you will be able to:

- (L1) define authentication vs. identification and quantify password strength via entropy;
- (L2) explain why passwords must be salted and hashed and evaluate storage schemes;
- (L3) compare MFA factors and quantify biometric trade-offs (FAR/FRR, EER);
- (L4) describe challenge–response and analyse Needham–Schroeder and its fix;
- (L5) trace Kerberos ticket exchange and reason about replay and freshness.

3.1 Passwords: Intuition to Formal Storage

Intuition. A password is a shared secret—like a clubhouse word. The server should recognise the word without storing it in plain text, because leaks happen. Instead it stores a one-way fingerprint.

Definition 3.1 (Entropy and guessing). If a password is drawn uniformly from N possibilities, its entropy is $H = \log_2 N$ bits. A dictionary of D candidates guesses correctly with probability $D/N = D \cdot 2^{-H}$.

For 8 random lowercase letters, $N = 26^8$, $H \approx 37.6$ bits. User-chosen words have far less: “p0ssw0rd” may be in a 10^6 -entry crack dictionary, so effective $H \approx 20$ bits.

Storing correctly. Never store pwd . Store $h = H(salt \parallel pwd)$ with a unique random $salt$ per user and a slow hash (bcrypt/Argon2). Salt defeats rainbow tables; slowness defeats brute force.

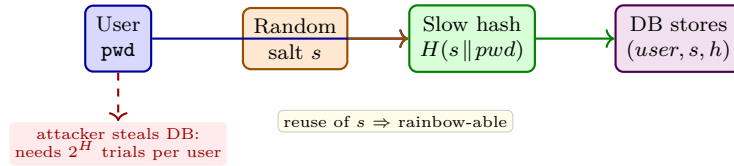


Figure 3.1: Password registration pipeline. Colour shows separation: blue secret, orange uniqueness, green one-wayness, violet stored verifier.

Listing 3.1: Toy salted hashing. Production: use bcrypt/Argon2 with work factor.

```

1 import hashlib, os
2 def register(pwd: str):
3     salt = os.urandom(16)                # 128-bit salt
4     h = hashlib.sha256(salt + pwd.encode()).hexdigest()
5     return salt.hex(), h                 # store (salt, h)
6
7 def verify(pwd: str, salt_hex: str, h: str) -> bool:
8     salt = bytes.fromhex(salt_hex)
9     return hashlib.sha256(salt + pwd.encode()).hexdigest() == h
10 # Attacker with DB still needs dictionary * slow-hash per salt
  
```

Offline guessing cost is $\approx D \times c_{\text{hash}}$; doubling hash time doubles attacker work. Online throttling and pepper ($HMAC_k$) add further defence.

Example 3.1 (How fast is cracking?). An 8-char random lowercase has $H = 37.6$ bits $\approx 2 \times 10^{11}$ possibilities. At 10^9 SHA-256/s (GPU) it falls in minutes; with bcrypt cost 12 (≈ 0.3 s per hash) it needs ≈ 2000 years. This is why “fast hash = broken storage” and OWASP mandates memory-hard Argon2id.

Definition 3.2 (Threat model for authentication). *Online guessing*: attacker tries logins against live server (rate-limited). *Offline cracking*: attacker stole $(salt, h)$ and tests $H(s \parallel \text{guess})$ locally—only slowness and entropy help. *Replay/eavesdrop*: attacker observes or replays messages; freshness (nonce/timestamp) is required.

Identification vs. authentication vs. authorisation. *Identification* claims “I am Alice” (a username). *Authentication* proves it (password/MFA). *Authorisation* decides what Alice may do—the subject of Ch. 4. Confusing these collapses the reference monitor.

3.2 MFA and Biometrics

Three factor classes: *know* (password/PIN), *have* (token/phone), *are* (fingerprint/face). MFA requires ≥ 2 distinct classes; two passwords is not MFA.

Definition 3.3 (Biometric errors). For threshold t , false accept rate $\text{FAR}(t) = P(\text{accept} \mid \text{impostor})$ and false reject $\text{FRR}(t) = P(\text{reject} \mid \text{genuine})$. The equal error rate EER is where $\text{FAR} = \text{FRR}$.

Tuning t trades security (low FAR) against usability (low FRR). No biometric is secret—you leave fingerprints everywhere—so biometrics must be paired with liveness checks and revocable templates.

Example 3.2 (Fingerprint vs. face). Fingerprint EER $\approx 1\%$ at phone sensors; face under varied lighting may be 2–5%. A data centre (10^4 entries/day, 1 impostor/year) cannot tolerate $\text{FAR} = 10^{-3}$ (10 false accepts/year) and pushes t toward low FAR, accepting higher FRR and adding a fallback (PIN). Phones do the opposite: low FRR for convenience, compensating with short retry lockouts.

Why MFA can still fail. SMS OTP is “have” but vulnerable to SIM-swap and SS7 interception—NIST 800-63B now discourages SMS. Push-based MFA suffers “MFA fatigue” where users approve spurious prompts. Strong MFA uses FIDO2/WebAuthn challenge–response with hardware keys: the “have” proves possession of a private key that never leaves the token.

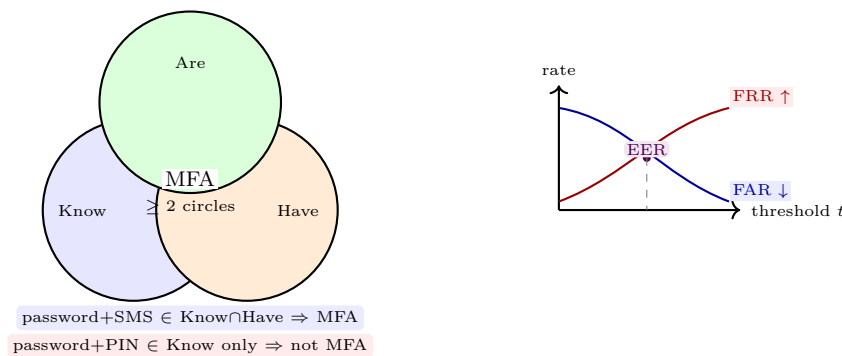


Figure 3.2: Left: MFA needs two distinct factor circles. Right: biometric threshold trades FAR against FRR; EER is the crossover.

3.3 Challenge–Response, Needham–Schroeder, Kerberos

Challenge–response. The verifier sends a fresh nonce N ; the claimant returns $F_K(N)$ (e.g. $\text{HMAC}_K(N)$). Freshness prevents replay. Without crypto, an eavesdropper replays.

Formal intuition. An authentication protocol must guarantee *aliveness* (peer recently participated), *freshness* (message is not a replay), and *binding* (key K_{AB} is bound to A and B , not an attacker). Needham–Schroeder aims to establish K_{AB} with those three, but misses freshness for B .

Listing 3.2: Toy challenge–response; real use: HMAC or signature.
Simple challenge–response (unilateral).

```

1 import os, hmac, hashlib
2 K = os.urandom(16) # pre-shared key
3 def verifier_issue(): return os.urandom(16) # nonce N
4 def prover_reply(N): return hmac.new(K, N, hashlib.sha256).digest()
5 def verifier_check(N, tag): return hmac.compare_digest(prover_reply(N), tag)
6 # Reflection attack? Attacker reflects N back; include identities: HMAC(K, "B||N")

```

Needham–Schroeder symmetric (1978). With key server S sharing K_{AS}, K_{BS} :

1. $A \rightarrow S : A, B, N_A$
2. $S \rightarrow A : \{N_A, B, K_{AB}, \{K_{AB}, A\}_{K_{BS}}\}_{K_{AS}}$
3. $A \rightarrow B : \{K_{AB}, A\}_{K_{BS}}$
4. $B \rightarrow A : \{N_B\}_{K_{AB}}$
5. $A \rightarrow B : \{N_B - 1\}_{K_{AB}}$

Steps 4–5 authenticate A to B via nonce challenge. Flaw: if old K_{AB} leaks, attacker replays message 3 and impersonates A (no freshness for B). Denning–Sacco fix adds timestamps/extra nonce: B checks recency.

Theorem 3.1 (Needham–Schroeder flaw). Without a freshness guarantee for B , compromise of a past session key K_{AB} lets an attacker re-authenticate as A indefinitely. Adding T_B or requiring B to consult S restores authentication.

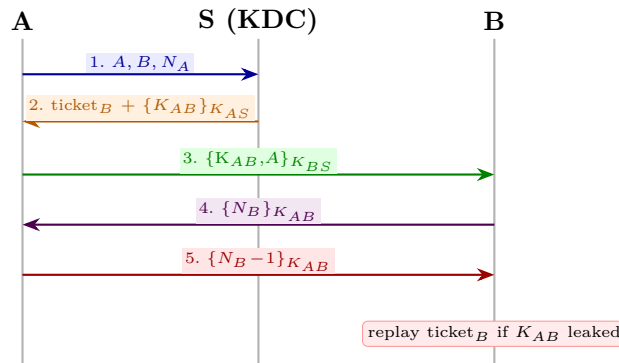


Figure 3.3: Needham–Schroeder symmetric-key exchange. Messages 4–5 are the nonce challenge–response; lack of freshness in message 3 enables replay.

Kerberos (Needham–Schroeder fix deployed). Realm with Authentication Server (AS) and Ticket-Granting Server (TGS). Timestamps replace nonces; tickets are $\text{TGT} = \{A, TGS, K_{A-TGS}, T_{exp}\}_{K_{TGS}}$, $\text{ST} = \{A, S, K_{A-S}, T_{exp}\}_{K_S}$. Flow:

Listing 3.3: Kerberos simplified flow (timestamps abbreviated).

```

1 # 1. Client -> AS : A, TGS, nonce
2 # 2. AS -> Client: {K_A_TGS, TGT}_K_A   where TGT={A,K_A_TGS,exp}_K_TGS
3 # 3. Client -> TGS: TGT, {A,time}_K_A_TGS, S
4 # 4. TGS -> Client: {K_A_S, ST}_K_A_TGS   where ST={A,K_A_S,exp}_K_S
5 # 5. Client -> S : ST, {A,time}_K_A_S
6 # 6. S -> Client: {time+1}_K_A_S           # mutual auth
7 authenticator = HMAC(K_session, A + str(timestamp))
8 # S checks |clock - timestamp| < 5min and ST not expired
  
```

Kerberos adds real fix: short-lived tickets (typically 8–10 h for TGT, minutes for ST), authenticators with timestamps, and a single sign-on. TGT lets Alice get STs without re-entering her password. Requires loosely synchronised clocks and careful replay cache. Mutual authentication comes from step 6: S proves it knows K_{A-S} by returning $\{time+1\}_{K_{A-S}}$.

Why timestamps and not more nonces? Nonces need a round-trip and server-side state (N pending). Timestamps are stateless: S checks recency via $|\text{clock} - \text{time}| < \Delta$ plus a replay cache of recent authenticators. The price is clock sync (Kerberos needs NTP within $\approx 5\text{min}$) and the cache must be bounded to avoid DoS.

Remark 3.1 (Public-key Needham–Schroeder and Lowe’s fix). The public-key variant ($A \rightarrow B : \{N_A, A\}_{K_B}$, $B \rightarrow A : \{N_A, N_B\}_{K_A}$, $A \rightarrow B : \{N_B\}_{K_B}$) falls to Lowe’s interleaving attack: attacker I posing as A to B reuses B ’s challenge on A . Fix: include responder identity, $B \rightarrow A : \{N_A, N_B, B\}_{K_A}$. BAN logic (Burrows–Abadi–Needham) later formalised such reasoning.

3.4 Exercises

Each exercise ties intuition (why it matters) to a verifiable artefact (number, proof, trace).

E3.1 Entropy maths. A policy requires 10 characters from 94 printable ASCII uniformly. Compute entropy in bits and expected guesses for an attacker trying 10^9 hashes/s with bcrypt cost 2^{12} . Contrast with a 4-word Diceware phrase (7776-word list).

E3.2 Salt and pepper. Explain why per-user salt stops rainbow tables but not dictionary attacks, and what a server-side pepper p (stored in HSM, used as $HMAC_p$) adds. What breaks if p is checked into git?

E3.3 Biometric tuning. A face system has FAR= 10^{-3} and FRR= 0.05 at threshold t_0 . At t_1 , FAR= 10^{-5} , FRR= 0.20. A building has 1000 staff entering twice daily and expects one impostor attempt per year. Compute expected false rejects and false accepts per year at each threshold. Which t would you pick and why?

E3.4 Needham–Schroeder replay. Assume attacker learned K_{AB} from a session last week and recorded $\{K_{AB}, A\}_{K_{BS}}$. Walk through how they impersonate A to B step-by-step. Show how adding $\{B, N_B\}_{K_{BS}}$ or a timestamp in message 3 blocks the attack.

E3.5 Kerberos clock skew. Kerberos allows 5-minute clock skew. An attacker replays a captured ticket ST with authenticator $\{A, \text{time}\}_{K_{A-S}}$ inside the window. How does the replay cache on S prevent this, and what denial-of-service risk does a large cache create? Propose a nonce-based alternative and its cost.

Chapter Notes

Morris–Thompson (1979) measured password guessability; Feldmeier–Karn (1990) and Boneh et al. catalogued Unix crypt evolution. Needham–Schroeder (1978) and Denning–Sacco (1981) framed freshness; Kerberos V5 (RFC 4120, Steiner–Neuman–Schiller) deployed the fix with tickets and timestamps—still the enterprise SSO backbone. For biometrics see Jain–Ross–Nandakumar and NIST 800-63B guidance on memorised secrets and MFA. FIDO2/WebAuthn (CTAP2) modernises challenge–response with public-key possession.

What to remember. Good authentication is not “clever passwords” but architecture: slow salted hashes, MFA with independent factors, freshness in every protocol step, and short-lived tickets. When you next design a login, sketch the attacker with the DB, then check each message for replay.

Chapter 4

Access Control: ACLs, RBAC, ABAC and Lattices

“Authentication tells you who they are; access control tells you what they may do.” — Every request passes a reference monitor that checks a policy. We build those policies from first principles.

From zero: no prior security model assumed. We start from a spreadsheet of who may do what, then fold it by columns (ACLs) and rows (capabilities), group by job (RBAC), generalise by attributes (ABAC), and finally order sensitivity in a lattice (Bell–LaPadula/Biba). Pictures first, then definitions, then code.

Learning Outcomes

After this chapter you will be able to:

- (L1) define the access matrix and derive ACLs and capability lists as dual views;
- (L2) model roles, hierarchies, and constraints in RBAC and decide when it fits;
- (L3) author ABAC policies over subject/object/environment attributes;
- (L4) draw a security lattice and apply BLP (no read-up, no write-down) and Biba;
- (L5) choose among ACL/RBAC/ABAC/capabilities for a given system and argue trade-offs.

4.1 The Access Matrix: Columns Are ACLs, Rows Are Capabilities

Intuition. Write a table: rows = subjects (Alice, Bob), columns = objects (fileA, printer), cells = rights $\{r, w, x\}$. This *access matrix* $M[S, O]$ is the conceptual truth; every real mechanism is a way to store it.

Definition 4.1 (Access matrix and its projections). $M : S \times O \rightarrow 2^R$ maps each subject–object pair to a set of rights. Column $M[\cdot, O]$ stored with O is an *ACL*; row $M[S, \cdot]$ held by S is a *capability list*. Both encode the same M .

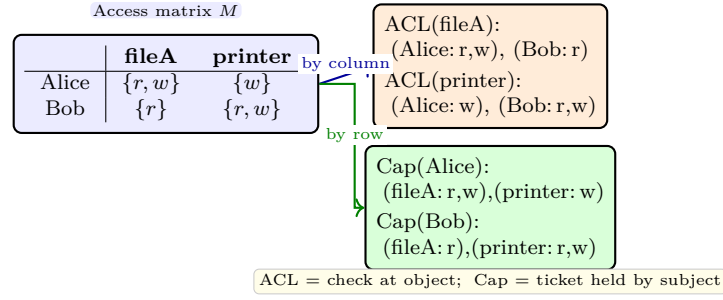


Figure 4.1: One abstract matrix, two storage duals. ACLs live with the object and are checked on access; capabilities live with the subject and are presented like tickets. Colour: blue truth, orange column view, green row view.

ACLs are natural when objects have owners (Unix `rw`x, NTFS ACLs) but checking “what can Alice access?” scans all ACLs. Capabilities answer that in $O(1)$ but need revocation (how to take a ticket back?)—indirection via a capability table or short expiry. Hybrid systems keep both.

Example 4.1 (Unix and Windows ACLs). Unix “`rw`x” for user/group/other is a compressed ACL with three entries; POSIX ACLs extend to `setfacl -m u:alice:rw fileA`. Windows NTFS stores per-file DACLs with SIDs and inheritance. Capabilities appear as file descriptors in Unix (if you hold fd 3 you can read) and as object handles in microkernels (seL4, Fuchsia).

Definition 4.2 (Reference monitor). A reference monitor is tamper-proof, always invoked, and small enough to verify (Neumann). It sits on every access path (Fig. 4.1 arrow) and consults the policy store; bypassing it breaks the model regardless of how clever the policy is.

Listing 4.1: ACL vs. capability checks (same matrix, dual lookup).

```

1 M = {("Alice", "fileA"):{'r', 'w'}, ("Alice", "printer"):{'w'},
2      ("Bob", "fileA"):{'r'},          ("Bob", "printer"):{'r', 'w'}}
3 def acl_check(obj, subj, op):      # object stores its column
4     acl = {s: r for (s, o), r in M.items() if o == obj}
5     return op in acl.get(subj, set())
6 def cap_check(subj, obj, op):      # subject holds its row (ticket)
7     caps = {o: r for (s, o), r in M.items() if s == subj}
8     return op in caps.get(obj, set())
9 assert acl_check("fileA", "Alice", 'w') == cap_check("Alice", "fileA", 'w')

```

4.2 RBAC: Roles as Indirection

Intuition. In a hospital, permissions follow the job, not the person. Alice *has* role Doctor, so she inherits read(chart), write(prescription). When Alice moves to Admin, we move one role assignment, not hundreds of ACL entries.

Definition 4.3 (RBAC₀–RBAC₃). Core RBAC₀: users U , roles R , permissions P , relations $UA \subseteq U \times R$ and $PA \subseteq P \times R$; a session maps a user to a subset of assigned roles. Hierarchy (RBAC₁) orders R by $\succeq$ (senior inherits junior). Constraints (RBAC₂) add separation of duty; RBAC₃ combines both.

Separation of duty: *static* (no user in both Purchaser and Approver) prevents fraud even off-session; *dynamic* (same session may not activate both) is more flexible. RBAC shines when roles are stable and auditable, but

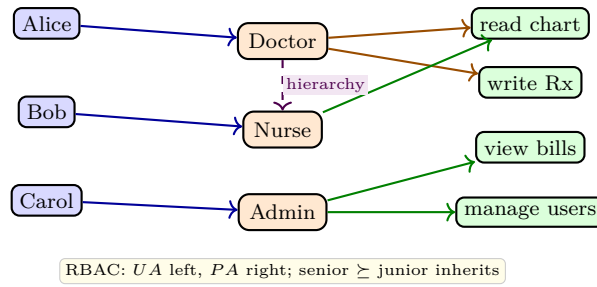


Figure 4.2: RBAC indirection. Users assigned to roles (UA), roles to permissions (PA). Changing a user’s job means one UA edit, not many PA edits. Dashed edge shows role hierarchy.

explodes when permissions need fine attributes (“doctors may read charts of patients in their ward between 9–5”)—that is ABAC territory.

Example 4.2 (Role explosion). A university with departments $\times$ courses $\times \{student, TA, instructor\}$ naïvely needs $D \cdot C \cdot 3$ roles. With hierarchy (Instructor $\succeq$ TA $\succeq$ Student per course) plus parameterised roles (Instructor(CS101)), count drops to $O(D + C)$. Without this, RBAC degenerates into per-user ACLs.

4.3 ABAC: Policies over Attributes

Intuition. Instead of naming roles, write a rule on *attributes*: subject attrs (dept, clearance), object attrs (owner, classification), environment attrs (time, location), and an action. A Policy Decision Point evaluates: permit iff $\text{policy}(\text{sub}, \text{obj}, \text{env}, \text{act}) = \text{true}$.

Definition 4.4 (ABAC policy). Let A_s, A_o, A_e be attribute sets. A policy is a predicate $P : A_s \times A_o \times A_e \times \text{Act} \rightarrow \{\text{permit}, \text{deny}\}$. The reference monitor calls P on every access; combining algorithms (deny-overrides, permit-overrides) resolve multiple policies.

Listing 4.2: ABAC policy: attribute rule with combining.

```

1 def policy(sub, obj, env, act) -> bool:
2     # Rule: doctors in same ward, during shift, may read charts
3     if sub["role"]=="doctor" and obj["type"]=="chart" and act=="read":
4         if sub["ward"]==obj["ward"] and 9 <= env["hour"] < 17:
5             return True
6     # Rule: owner may always read own record
7     if sub["id"]==obj["owner"] and act=="read":
8         return True
9     return False # default deny (closed policy)
10
11 # PDP: deny-overrides if any deny policy triggers; here single policy
12 def pdp(sub, obj, env, act): return "permit" if policy(sub, obj, env, act) else "deny"
  
```

ABAC generalises ACL and RBAC: an ACL is “attr = identity”, a role is one subject attribute. Price: policy review is harder (attribute sprawl, conflicting rules, time-of-check–time-of-use). XACML/OPA/Rego are industry policy languages; minimal, testable, version-controlled policies are essential.

Remark 4.1 (Combining and default deny). With multiple rules, *deny-overrides* (any deny $\Rightarrow$ deny) is safest for closed policies; *permit-overrides* suits open collaborations. NIST 800-162 recommends a default deny and explicit permit rules, plus a *break-glass* override (audited emergency permit) that temporarily bypasses deny.

4.4 Lattice Model: Classifying Information Flow

Intuition. Some data is more sensitive than other data: Public $\sqsubseteq$ Confidential $\sqsubseteq$ Secret $\sqsubseteq$ Top Secret, and compartments add dimensions (e.g. Crypto, Nuclear). Sensitivity forms a *lattice*: any two labels have a least upper bound (join $\sqcup$) and greatest lower bound (meet $\sqcap$). Information may only flow *up* the lattice.

Definition 4.5 (Security lattice). A lattice $(L, \sqsubseteq)$ is a partial order where every pair a, b has $\sup\{a, b\} = a \sqcup b$ and $\inf\{a, b\} = a \sqcap b$. Labels are $\ell = (\text{level}, \text{compartments})$ with $\ell_1 \sqsubseteq \ell_2$ iff $\text{level}_1 \leq \text{level}_2$ and $\text{compartments}_1 \subseteq \text{compartments}_2$.

Two classic uses read the lattice in opposite directions:

- **Bell–LaPadula (confidentiality):** no read-up, no write-down. Subject S with clearance $\lambda(S)$ may read object O only if $\lambda(O) \sqsubseteq \lambda(S)$ (Simple Security) and write only if $\lambda(S) \sqsubseteq \lambda(O)$ (Star Property). Prevents leaking Secret into Public.
- **Biba (integrity):** dual—no read-down, no write-up. Low-integrity data must not contaminate high-integrity decisions.

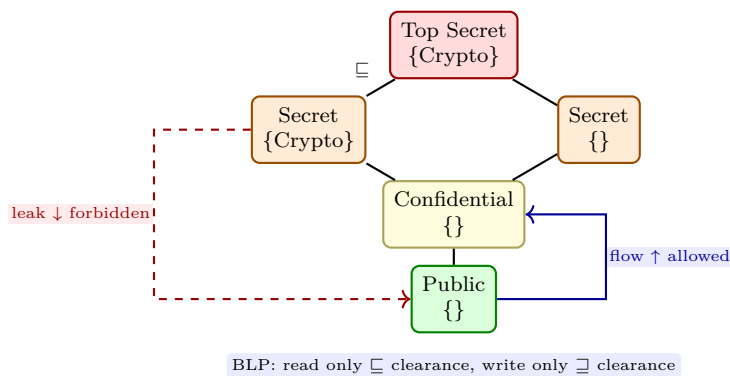


Figure 4.3: Hasse diagram of a security lattice (levels $\times$ compartments). Information flows upward; BLP forbids downward flow (red dashed). Join of the two Secret nodes is the TS node. The diagram is a “ravioli” stacking of levels with compartment sets.

Theorem 4.1 (BLP enforces no downward leak). If all reads satisfy $\lambda(O) \sqsubseteq \lambda(S)$ and writes satisfy $\lambda(S) \sqsubseteq \lambda(O)$, and labels are monotone (tranquillity: labels do not change to leak), then no sequence of operations copies data from higher to strictly lower label.

Real systems relax strict BLP with *trusted subjects* (downgraders) that are verified to sanitise, and handle covert channels (timing, resource exhaustion) outside the lattice. Multilevel databases and SELinux MLS/MCS implement lattice ideas at OS granularity.

Example 4.3 (Covert channel outside the lattice). BLP stops explicit read/write, but a Top Secret process can signal one bit to a Public process by modulating CPU load (high = 1, low = 0) if they share a scheduler. The data *flow* is not via a labelled object, so the lattice does not see it. Mitigation: partition resources, add noise, or bound channel capacity (Lampson, 1973).

4.5 Exercises

- E4.1 **ACL vs. capability audit.** Given M with 1000 subjects and 10^5 files averaging 10 entries per ACL, compute cost of “list all files Alice can write” via ACL scan vs. capability cache. Propose a hybrid that keeps ACLs authoritative but caches per-subject lists, and explain revocation.
- E4.2 **RBAC separation of duty.** Design RBAC for a bank: roles Teller, Auditor, Manager with permissions post/withdraw/audit/approve-loan. State static and dynamic SoD constraints that prevent (a) self-audit and (b) single-person loan fraud. Show a violating UA assignment.
- E4.3 **ABAC policy authoring.** Write an ABAC rule: “Nurses may write vitals for patients in their ward during their shift, and patients may read their own chart anytime.” Identify subject/object/environment attributes, give allow/deny test cases, and show a conflict where both an allow and deny rule fire; apply deny-overrides.
- E4.4 **Lattice join/meet.** For $L = \{Public, Confidential, Secret\} \times \mathcal{P}(\{Med, Fin\})$ ordered by level and $\subseteq$ on compartments, compute $(Secret, \{Med\}) \sqcup (Confidential, \{Fin\})$ and $\sqcap$, and list all labels $\sqsubseteq (Secret, \{Med, Fin\})$.
- E4.5 **BLP vs. Biba trade-off.** A system must enforce both confidentiality (BLP) and integrity (Biba) on the same lattice. Show that the combined rules forbid any read or write between distinct levels, and propose how real systems (e.g. SELinux) handle this by separating confidentiality and integrity lattices or using trusted subjects.

Chapter Notes

Lampson (1971) introduced the access matrix and its ACL/capability duality; Saltzer–Schroeder (1975) framed the reference monitor. RBAC was systematised by Ferraiolo–Kuhn (1992) and standardised as ANSI INCITS 359 (RBAC_{0–3}). ABAC generalises both and underpins NIST 800-162 and OPA/Rego. Denning (1976) formalised lattices for information flow; Bell–LaPadula (1973, MITRE) and Biba (1977) are the confidentiality/integrity duals. For SELinux MLS see Mayer–MacMillan–Caplan; for capabilities see Levy (1984) and modern Capsicum/CHERI.

Chapter 5

Software Security: Buffer Overflows, Injection, and Hardening

“Software is the attacker’s favourite battlefield: every unchecked input is an invitation.” — From memory corruption to injected code, the flaw is the same: trusting data as if it were control.

From zero: what this chapter builds on. We assume no prior security knowledge. We start from how a program lays out memory (the stack), what it means to “overflow” a buffer, and why treating user input as code is dangerous. Every attack is shown first as a picture and concrete input-output, then formalized, then hardened. No cryptography or OS prerequisite beyond Ch. 1 foundations.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain the stack layout and how an unchecked copy overwrites the return address;
- (L2) construct a minimal buffer-overflow exploit and describe its payload structure;
- (L3) distinguish SQL injection and XSS (stored, reflected, DOM) with concrete payloads;
- (L4) explain stack canaries, ASLR, and DEP/W^X and how each blocks an exploit stage;
- (L5) apply input validation and safe-API principles to prevent injection and overflow.

5.1 Why Software Fails: Trusting Data as Control

Every program mixes *data* (what the user typed) with *control* (where to jump, what query to run, what script to render). The processor, the SQL engine, and the browser all fetch the next instruction from somewhere writable if the programmer was careless. Think of a paper form: the box for “name” is data, but if you write past the box and onto the “approved” checkbox, data has become control.

Formally, a *software vulnerability* is a state where an input causes execution to diverge from the intended

control-flow graph or access-control policy. The three families below are the classic instances: memory corruption (control hijack), SQL injection (query-structure hijack), and XSS (browser-script hijack).

A useful mental model: the program is an interpreter that reads two tapes, *program* and *input*. A vulnerability merges the tapes so input symbols are decoded as program symbols. Hardening separates them again.

Definition 5.1 (Control vs. data). *Control data* determines what executes next (return addresses, function pointers, SQL parse tree, HTML parser state). *Non-control data* only influences values. Attacks that corrupt control data are strictly more powerful than those that only leak values.

5.2 Buffer Overflows: Overwriting the Stack

On x86-like call stacks the frame for a function contains, from low to high addresses: local buffers, saved frame pointer, and the *return address* — the address execution resumes after `return`. The stack grows toward lower addresses, but buffers fill upward, toward control data.

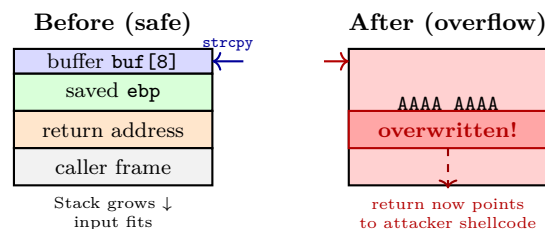


Figure 5.1: Stack frame before and after an overflow. An 8-byte buffer receives 18 bytes; the extra bytes overwrite the saved frame pointer and return address. Color shows corrupted region in red.

Definition 5.2 (Buffer overflow). A *buffer overflow* occurs when a program writes k bytes into a b -byte object with $k > b$ and without bounds checking, overwriting adjacent memory. If adjacent memory holds control data (return address, function pointer), the write is a *control-flow hijack*.

```

1 void vulnerable() {
2     char buf[8];
3     gets(buf);           // no length check -- NEVER use
4     printf("echo: %s\n", buf);
5 }
6 int main(){ vulnerable(); return 0; }
```

Input `AAAAAAAAAAAA\xb6\x84\x04\x08` (12 As + little-endian address) overwrites the return address with `0x080484b6`. When `vulnerable` returns, `%eip` jumps to the attacker address instead of `main`.

Exploit anatomy (intuition $\rightarrow$ formal):

- Overflow*: supply $b + \Delta$ bytes where Δ reaches the return slot.
- Payload*: shellcode or ROP gadgets placed in buffer or known library.
- Redirect*: overwritten return address = payload address.

Modern payloads avoid null bytes (which terminate `strcpy`) and use *NOP sleds* (`\x90 sled`) to tolerate address imprecision: landing anywhere in the sled slides to shellcode.

Example 5.1 (Off-by-one and 64-bit). `for(i=0;i<=8;i++) buf[i]=input[i];` writes 9 bytes into `buf[8]`. On 32-bit little-endian this corrupts the saved `ebp` LSB, shifting the next frame. On 64-bit the canary and 8-byte addresses make exploitation harder but not impossible; ROP is the norm because stack is non-executable.

Remark 5.1 (Heap and other targets). Not every overflow hits the stack. Heap overflows corrupt heap metadata or adjacent objects (e.g., C++ vtables, function pointers in structs). The principle is identical: data written past its object reaches control data. We focus on the stack because it is the clearest illustration.

5.3 Injection I: SQL Injection

Databases execute queries assembled by string concatenation. If user input is concatenated verbatim, it can escape the data context and become SQL syntax.

```

1 # Vulnerable login (Python DB-API style)
2 query = "SELECT * FROM users WHERE name = '" + username + "' AND pw = '" + password + "'"
3 cursor.execute(query)

```

Attacker sets `username = ' OR 1=1; -` yielding:

```
SELECT * FROM users WHERE name = " OR 1=1; -' AND pw = "
```

The `-` comments out the remainder; `1=1` is always true, so every row is returned — authentication bypassed.

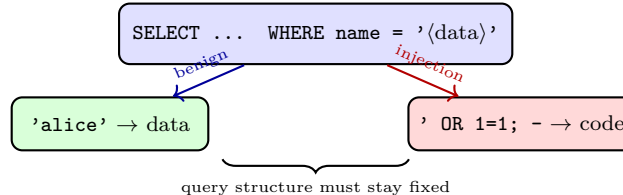


Figure 5.2: SQL injection as breaking out of the data context. Benign input stays inside the quoted literal (blue→green); attacker input escapes it and alters the query tree (red).

Definition 5.3 (SQL injection (SQLi)). *SQLi* is an injection where untrusted input is interpolated into a SQL statement without parameterisation, allowing the attacker to change the parse tree of the query.

Blind and second-order variants exfiltrate data bit-by-bit (`AND SUBSTRING(pw,1,1)='a'`) or store payloads that fire later when an admin views them. The fix is invariant: never build SQL by string concatenation with untrusted data.

```

1 # Safe: structure fixed, data bound separately (prepared statement)
2 cursor.execute("SELECT * FROM users WHERE name = ? AND pw = ?", (username, password))
3 # The template is compiled first; bindings are typed data, never parsed as SQL.

```

Example 5.2 (UNION extraction). With `cat=' UNION SELECT password FROM users-` the query becomes `SELECT id FROM products WHERE cat=" UNION SELECT password FROM users-`. The second `SELECT` appends password hashes to the product list, exfiltrating them in the normal response.

5.4 Injection II: Cross-Site Scripting (XSS)

XSS injects JavaScript that the victim's browser executes in the origin of the vulnerable site — stealing cookies, session tokens, or performing actions as the user. Unlike SQLi, the victim is the user, not the database.

Stored XSS Payload is saved on the server (comment, profile) and served to every visitor. Example `<script>alert(1)</script>` persists (the tag executes when the page is rendered).

Reflected XSS Payload arrives in the request (e.g., `?q=<script>...`) and is echoed verbatim in the response. Needs a lure URL.

DOM XSS No server reflection; client-side JS unsafely sinks attacker-controlled data into `innerHTML` or `eval`. Example: `e1.innerHTML = location.hash.slice(1)`.

Definition 5.4 (XSS). *Cross-Site Scripting* occurs when an application includes untrusted data in an HTML/JS context without proper output encoding, causing the browser to execute it as script in the application's origin.

Example 5.3 (Reflected payload). Server echoes: `<p>Search: $q</p>` without encoding. Attacker crafts `https://site/search?q=<svg onload=alert(1)>`. Victim clicks; browser parses `<svg...>` as active content. Stored variant: attacker posts a profile name `<img src=x onerror=alert(1)>` that every viewer executes.

Output encoding is context-sensitive: HTML-entity encode in element content, attribute-encode in attributes, JS-encode inside `<script>`. A single `htmlspecialchars` is not universally sufficient.

```

1 # Context-aware encoding (Python illustration)
2 import html
3 safe_html = html.escape(user_input)           # for <div>content</div>
4 safe_attr = html.escape(user_input, quote=True) # for <div title="...">
5 # For JS contexts use json.dumps; for URLs use urllib.parse.quote

```

Content Security Policy (CSP) adds a second layer: `Content-Security-Policy: script-src 'self'` blocks inline scripts even if an injection slips through.

5.5 Hardening: Canaries, ASLR, DEP, and Input Validation

No single defence suffices; they compose in layers. Each blocks a different stage of the exploit chain, from input entry to code execution.

Stack canaries A random secret word is placed between buffer and return address. On function return the canary is checked; mismatch aborts via `__stack_chk_fail`. Bypass requires leaking or brute-forcing the canary or corrupting elsewhere (e.g., heap, function pointer before the canary).

ASLR Randomises base addresses of stack, heap, libraries on each run (and with PIE, the executable itself). Attacker cannot hard-code gadget addresses; needs an information leak or brute force (weak on 32-bit with only 8–16 bits of entropy; strong on 64-bit with 28+).

DEP / W^X Marks data pages non-executable (NX bit); injected shellcode will fault on fetch. Bypass via *return-oriented programming* (ROP): chain existing executable gadgets ending in `ret` to achieve arbitrary

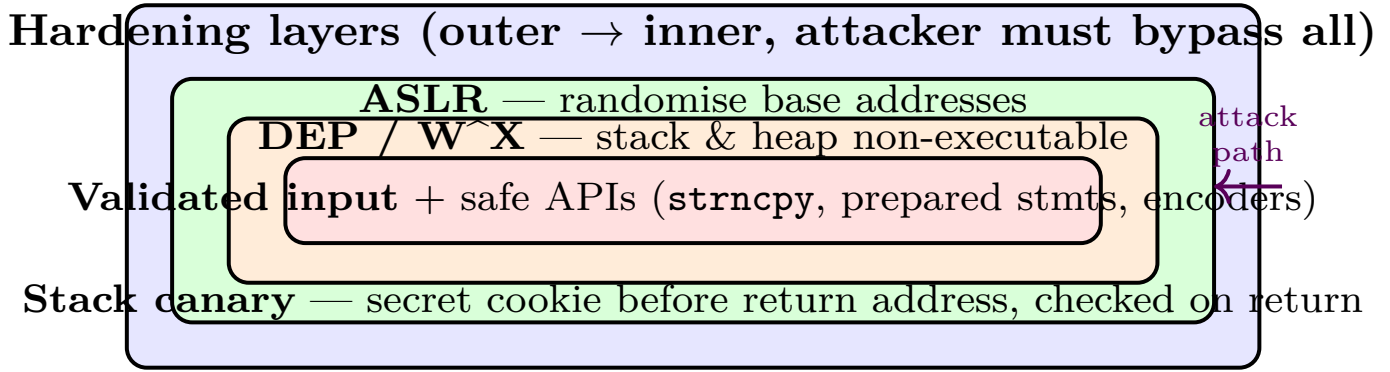


Figure 5.3: Defence in depth for memory and injection. Each layer blocks a different exploit stage; none alone is sufficient.

computation without injecting code.

Input validation Allow-list validation: accept only known-good patterns (type, length, charset, range); reject or encode the rest. For memory, use bounds-checked APIs (`fgets`, `strncpy`, `snprintf`, `gets` never); for SQL, prepared statements; for HTML, context-aware encoding and CSP.

Composition matters: canary stops linear overflows, ASLR stops address reuse, DEP stops code injection, validation stops the malformed input from entering at all. Together they force the attacker to chain multiple independent bypasses — a leak to defeat ASLR, a ROP chain to defeat DEP, and a canary bypass to reach the return address.

Theorem 5.1 (Layering principle). No single hardening mechanism restores full memory safety; only their composition (plus safe languages or formal verification) approaches it. Each layer raises exploit cost by requiring an additional primitive (leak, write, execute).

5.6 Exercises

- E5.1 Stack arithmetic.** A function has `char buf[16]`; then saved `ebp` (4 B) then return address (4 B). (a) How many bytes must an input contain to overwrite the first byte of the return address? (b) If a 4 B canary is placed after `buf`, how many bytes to reach it, and what must the attacker do to avoid detection?
- E5.2 Exploit the C code.** Given the `vulnerable()` listing in §5.2, craft a concrete 24-byte payload that overwrites the return address with `0x0804851a` on little-endian x86, explaining each segment (filler, canary if any, address bytes, null-byte issues).
- E5.3 SQLi construction.** A query is `SELECT id FROM products WHERE cat='$cat' ORDER BY price`. Construct a payload for `$cat` that (a) returns all rows, (b) extracts the first character of the admin password via a blind boolean condition, (c) propose the single-line fix using parameterised queries.
- E5.4 XSS taxonomy.** For each scenario classify as stored, reflected, or DOM XSS and give the precise sink: (a) comment `<b>$name</b>` stored unescaped, (b) `?error=$msg` echoed, (c) `location.hash` assigned to `innerHTML`. For (c) propose a safe alternative using `textContent`.
- E5.5 Defence bypass reasoning.** Explain why DEP alone does not stop ROP and why ASLR alone does not

stop a leak-assisted attack. Describe a two-stage bypass that defeats DEP+ASLR together (information leak $\rightarrow$ ROP) and which additional layer (canary or validation) would block it earliest.

Chapter Notes

Aleph One, “Smashing the Stack for Fun and Profit” (Phrack 49, 1996) is the canonical overflow introduction; modern defences are surveyed in Szekeres et al., “SoK: Eternal War in Memory” (Oakland 2013). SQLi taxonomy follows Halfond et al. (2006); XSS contexts and encoders are detailed in OWASP Cheat Sheets and Klein (2005). For ROP see Shacham (CCS 2007). The hardening triad (canaries/StackGuard, ASLR/PaX, DEP/W^X) is standard in all modern OSes and extended by Control-Flow Integrity (Abadi et al., 2005).

Chapter 6

Network Security: Firewalls, IDS/IPS, TLS and VPNs

“The network is the adversary’s highway.” — Packets cross untrusted hops; we decide what crosses the boundary, what we notice, and how we keep it confidential while in transit.

From zero: where we are in the story. Ch. 5 secured the program; now we secure the wire. Every packet is an untrusted letter: the header is the envelope, the payload is the contents. This chapter asks four questions from first principles: what to drop at the gate (firewalls), what to watch for inside (IDS/IPS), how to whisper so only the intended reader hears (TLS), and how to build a private tunnel over a public road (VPNs).

Learning Outcomes

After this chapter you will be able to:

- (L1) compare packet-filter, stateful, and application-layer firewalls and write a rule set;
- (L2) contrast signature-based vs. anomaly-based detection and explain IDS vs. IPS placement;
- (L3) describe the TLS 1.3 handshake, key schedule, and why it provides forward secrecy;
- (L4) distinguish IPSec (tunnel vs. transport) from SSL/TLS VPNs and their use cases;
- (L5) reason about rule ordering, false positives/negatives, and the trade-off between visibility and encryption.

6.1 The Network Boundary

A *perimeter* separates a trusted interior (enterprise LAN) from an untrusted exterior (Internet). Middleboxes at the perimeter enforce policy on flows defined by the 5-tuple (srcIP, dstIP, srcPort, dstPort, proto). Two complementary ideas apply everywhere in this chapter: *default deny* (drop unless explicitly allowed) and *least privilege* (allow the narrowest flow that still serves the need).

Intuition: the envelope (header) says where a letter comes from, goes to, and what service it claims. The

content (payload) may be encrypted. A firewall reads envelopes; an IDS may open copies; TLS seals the content so only the recipient can open it.

6.2 Firewalls: Packet Filter, Stateful, Application

Picture three checkpoints of increasing understanding. Each sees strictly more context than the previous, at higher cost.

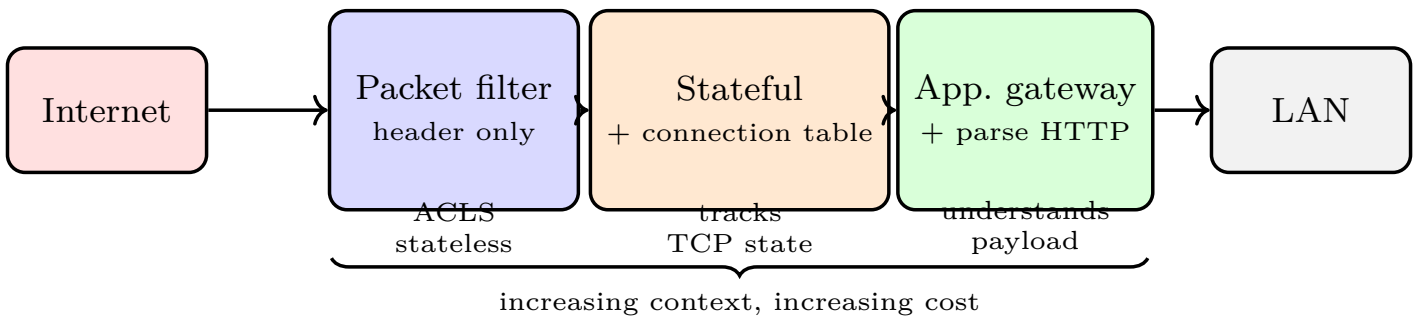


Figure 6.1: Firewall generations as a pipeline: each stage sees more context. Packet filters are fast but blind to state; stateful firewalls add a connection table; application gateways proxy and parse the payload.

Packet filter (stateless) Rules match header fields only. Example ACL: `deny tcp any any eq 23` (block Telnet). Fast, cheap, but cannot tell whether a packet belongs to an established flow; must allow broad reverse rules. Implements at line rate in routers.

Stateful firewall Maintains a *state table* of active connections (SYN seen, ESTABLISHED, RELATED). Return traffic is allowed only if it matches an existing entry. E.g., allow outbound SYN, then allow inbound SYN-ACK only if a matching SYN is in the table — blocks unsolicited inbound TCP and spoofed ACKs.

Application-layer gateway / NGFW Terminates the connection, parses the application protocol (HTTP, DNS, SMTP), and enforces content policy: `deny http POST to /admin from !corp`, strip ActiveX, enforce TLS inspection with consent. Highest visibility, highest latency and certificate-management cost. Often called next-generation firewall (NGFW) when combined with IDS.

Example 6.1 (Rule order matters). Default-deny with three rules in order: (1) `allow tcp 10.0.0.0/8 any eq 443`, (2) `allow tcp any 10.0.0.5 eq 80`, (3) `deny ip any any`. A packet to 10.0.0.5:80 matches (2); if (2) were below (3) it would be dropped. First-match semantics and shadowing are a leading cause of misconfiguration. Tools like `iptables -L -n -line-numbers` help audit order.

Placement: packet filter at the edge router for coarse drops, stateful firewall at the perimeter, app gateway in front of web farms. Egress filtering (controlling outbound) catches compromised hosts calling home — often more valuable than ingress rules.

Remark 6.1 (DMZ). A *demilitarised zone* hosts public-facing servers between two firewalls: outer allows Internet → DMZ, inner allows DMZ → LAN only for explicitly needed flows (e.g., app → DB on 5432). Compromise of the DMZ does not directly expose the LAN.

6.3 IDS and IPS: Signature vs. Anomaly

Definition 6.1 (IDS vs. IPS). An *Intrusion Detection System* (IDS) is passive: it copies traffic (SPAN port or TAP), raises alerts, does not block. An *Intrusion Prevention System* (IPS) is inline: it can drop, reset, or quarantine. Every IPS is an IDS that also enforces; placement determines risk of false positives becoming outages.

Two detection philosophies compete.

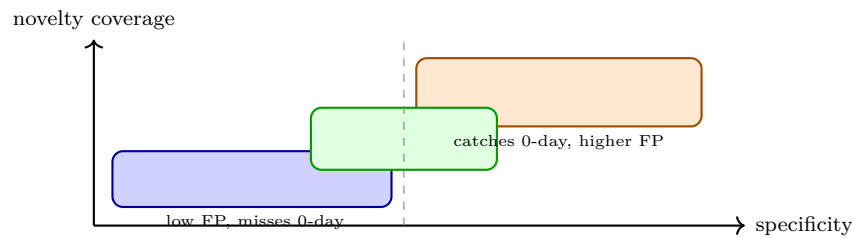


Figure 6.2: Detection trade-off. Signatures are precise but blind to novel attacks; anomaly detectors cover the unknown at the cost of more false positives. Production systems hybridise.

Signature-based Matches known patterns: byte sequences, regex on headers/payload, or Snort-style rules (alert tcp any any -> any 80 (msg:"SQLi"; content:"'" OR 1=1";)). Low false positives, zero-day blind, needs frequent updates. Analogous to anti-virus signatures.

Anomaly-based Learns a model of normal (traffic volume, n-grams, entropy, host behaviour) and flags deviation beyond a threshold. Catches novel attacks; suffers false positives and adversarial evasion (mimicry: attacker shapes traffic to look normal). Needs clean training data.

```

1 # Toy signature match (IDS-style inspection)
2 import re
3 RULES = [re.compile(pat) for pat in [r"' OR 1=1", r"<script", r"\.\."/]]
4 def inspect(payload: str):
5     for r in RULES:
6         if r.search(payload):
7             alert(f"signature hit: {r.pattern!r} in {payload[:80]}")
8             return "ALERT"
9     return "OK"
10 # Anomaly sketch: flag if byte-entropy deviates >3 sigma from baseline

```

Metrics: true/false positives/negatives, ROC curve. Tuning moves the threshold along the curve — blocking (IPS) demands low false positives, so signatures dominate inline; anomaly is often alert-only for SOC analysts. Encrypted traffic blinds payload inspection, pushing detection to metadata (flow sizes, timing) and endpoints.

Placement matters: network IDS watches the wire, host IDS watches syscalls and logs. Modern EDR/XDR fuses both. An IPS that false-positives on legitimate traffic creates a self-inflicted denial of service.

6.4 TLS 1.3 Handshake: Whispering in Public

TLS provides confidentiality, integrity, and server (optionally client) authentication over an untrusted channel. Version 1.3 (RFC 8446) removed legacy features that caused a decade of attacks.

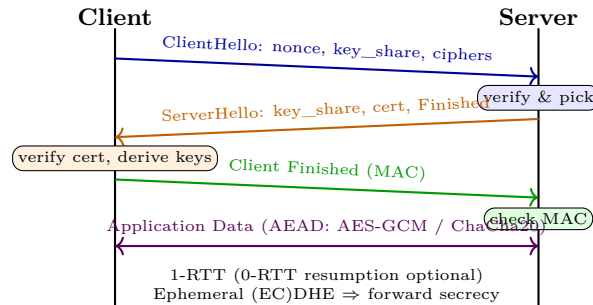


Figure 6.3: TLS 1.3 handshake (simplified): one round trip negotiates an ephemeral Diffie–Hellman share and authenticates the server. The Finished MACs bind the transcript; everything after is AEAD-encrypted.

Key ideas (intuition → formal): ephemeral (*EC*)DHE means a fresh Diffie–Hellman exponent is generated per connection; the session key is g^{xy} for client secret x and server secret y . Compromise of long-term signing keys later does not reveal x or y , so past sessions stay secret — *forward secrecy*. Contrast with RSA key transport (pre-1.3) where the session key was encrypted under the server’s long-term public key: later key compromise decrypts all past sessions.

The handshake is one round trip (1-RTT); 0-RTT resumption replays a previously established PSK for speed at replay risk, so servers must enforce anti-replay (single-use tickets, freshness checks).

Definition 6.2 (TLS 1.3 properties). *Authenticated key exchange* with forward secrecy; *record layer* uses AEAD (AES-GCM or ChaCha20-Poly1305) for confidentiality + integrity; cipher agility is removed (only 5 suites). Handshake is encrypted after ServerHello, hiding certificates from passive observers.

Certificates bind server identity to a public key via a CA chain. Client validates the chain, hostname (SAN), and expiry before deriving keys. Downgrade protection (special random values) and the **Finished** transcript MAC prevent version/cipher rollback attacks.

Example 6.2 (What 1-RTT means). Client sends key_share g^x speculatively without knowing the server’s group. If the server prefers a different group, it sends **HelloRetryRequest** — costing an extra RTT. Implementations therefore send shares for the two most likely groups (x25519, P-256).

6.5 VPNs: Private Tunnels over Public Roads

A *Virtual Private Network* creates a logically private link over a public network by tunnelling and encrypting. The tunnel is a virtual point-to-point link; routing decides what enters it.

IPSec Network-layer suite: *AH* (integrity only, now rare) and *ESP* (confidentiality + integrity). Two modes: *transport* (encrypts payload, keeps original IP header — host-to-host) and *tunnel* (encapsulates the entire IP packet in a new IP header — gateway-to-gateway). Key management via *IKEv2*. Used for site-to-site links and always-on corporate backhaul. Kernel-level, one SA per flow.

SSL/TLS VPN Application-layer tunnel over TLS (often via a browser or thin client). Easier to deploy through NAT/firewalls (runs over TCP 443), supports fine-grained per-application access. Two flavours: *portal* (reverse-proxy to web apps) and *tunnel* (TUN/TAP adapter carrying IP). Preferred for remote-access and BYOD; user-space, firewall-friendly.

Think of IPsec tunnel as putting the entire letter (with its envelope) inside a new outer envelope addressed gateway-to-gateway; transport mode only seals the letter contents. TLS VPN is like accessing the office through a single HTTPS door that proxies you inside.

Example 6.3 (Tunnel vs. transport on the wire). Two gateways linking 10.1.0.0/16 and 10.2.0.0/16 use IPsec *tunnel* mode: packet [IP_{inner}: 10.1.0.5→10.2.0.9 | TCP] becomes [IP_{outer}: GW_A→GW_B | ESP | IP_{inner} | TCP] on the wire. Two laptops talking directly could use *transport* mode with no outer header.

Comparison: IPsec excels at site-to-site (always on, transparent to apps); TLS VPN excels at road warriors (NAT traversal, per-app policy). Neither replaces endpoint security — a compromised laptop inside the tunnel is still inside the perimeter. Split tunnelling (only corporate traffic through VPN) trades security for bandwidth and must be policy-controlled.

	IPSec	TLS VPN
Layer	Network (L3), kernel	Transport/App (L4/L7), user-space
Through NAT	Hard (ESP, needs NAT-T)	Easy (TCP 443)
Client	Heavy (IKEv2, SA per flow)	Light (browser / thin agent)
Granularity	Per-host / per-subnet	Per-application / per-URL
Best for	Site-to-site, always-on	Remote access, BYOD, extranet

Table 6.1: IPsec vs. TLS VPN at a glance. Choice is driven by who connects and what middleboxes lie in between.

Remark 6.2 (Why this all composes). Firewalls decide *whether* a packet may pass, IDS/IPS decides *whether it looks malicious*, TLS decides *whether anyone in the middle can read it*, and VPN decides *whose network it appears to be on*. A packet from a road warrior therefore traverses: hotel NAT → TLS VPN (encrypted) → perimeter firewall (state checked) → IDS (signature scan on decrypted copy) → internal server over TLS. No single box sees everything; defence in depth does.

Definition 6.3 (Security association (SA)). An *SA* is the simplex state for one direction of an IPsec tunnel: encryption/integrity algorithms, keys, sequence counter, and lifetime. IKEv2 negotiates two SAs (inbound/outbound) per flow; rekeying before expiry preserves forward secrecy of the data channel.

6.6 Exercises

E6.1 Firewall rules. Write a default-deny ACL for a web server 203.0.113.10 that (a) allows inbound TCP 443 and 80 from anywhere, (b) allows outbound DNS (UDP 53) to 8.8.8.8, (c) blocks Telnet (TCP 23) explicitly, (d) drops all else. Explain why the DNS rule must be stateful or paired with a reverse rule.

E6.2 Stateful vs. stateless. A stateless filter allows `tcp any any established`. Show how an attacker can abuse this to reach an “established-only” internal service by setting the ACK bit. How does a true state table prevent it? Give the exact table entry needed after a legitimate SYN.

- E6.3 **IDS trade-offs.** A signature set has 1% false-positive rate on 1M daily events with 10 true attacks. Compute expected alerts and precision. An anomaly detector catches 9/10 attacks at 5% FPR. Compute its alerts and explain which is suitable for IPS blocking vs. IDS monitoring, using ROC intuition.
- E6.4 **TLS 1.3 reasoning.** (a) Why does ephemeral (EC)DHE give forward secrecy while RSA key transport does not? (b) What does the **Finished** MAC cover and which attack does it prevent? (c) Why is 0-RTT replayable and what server-side mitigation is required?
- E6.5 **VPN design.** A company needs (i) permanent link between two data centres and (ii) remote access for 500 BYOD laptops through hotel NATs. For each, choose IPsec vs. TLS VPN, mode (tunnel/transport/portal), and justify with firewall traversal, client provisioning, and granularity.

Chapter Notes

Firewalls follow Cheswick–Bellovin (1994) and the “default deny” doctrine; NGFW adds DPI. IDS/IPS taxonomy is Denning (1987) and Axelsson (2000); Snort/Suricata exemplify signatures, while Sommer–Paxson (2010) critiques anomaly hype. TLS 1.3 is RFC 8446 (Rescorla, 2018); prior attacks (BEAST, POODLE, Heartbleed) motivated its simplification. IPsec is RFC 4301/7296 (Kent, Kaufman); SSL VPN trade-offs are surveyed in Frankel et al. (NIST SP 800-77).

Chapter 7

Operating System Security: Isolation and Containment

“The operating system is the reference monitor that never sleeps.” — Every application trusts the OS to keep its memory private, its files intact, and its privileges bounded. This chapter asks how that trust is built: by isolating, mediating, and containing.

From zero: we build here, no OS course needed. We assume you know what a process and a file are at an everyday level (a running program, a named blob of bytes). From that we build isolation, privilege, sandboxing, and virtualization from first principles—pictures before formalism, then definitions, examples, and analysis.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain why isolation is the foundation of OS security and state the three reference-monitor properties;
- (L2) describe privilege separation, privilege bracketing, and the principle of least privilege with concrete mechanisms;
- (L3) compare sandboxing techniques (system-call filtering, capability-based, language-based);
- (L4) distinguish hypervisor types, VM isolation guarantees, and VM-escape threats;
- (L5) analyse container isolation (namespaces, cgroups, seccomp) and contrast containers with VMs.

7.1 Why the OS Must Isolate

On a single machine dozens of programs run at once: your browser, a PDF reader opening an untrusted attachment, a background updater. Without isolation a bug in one becomes a compromise of all. Picture an

apartment building: each flat has its own lock and walls; the hallway (kernel) mediates who may enter where. Formally, *isolation* means a fault or compromise in compartment *A* cannot read, write, or control resources of compartment *B* except through an explicit, mediated channel. Two design principles follow immediately:

Definition 7.1 (Least privilege & complete mediation). *Least privilege*: every component holds only the privileges it needs to do its job, for the shortest time needed. *Complete mediation*: every access to a protected resource is checked.

Without least privilege a compromised renderer can read `/etc/shadow`; without complete mediation a check that can be bypassed is no check at all.

Example 7.1 (Threat model without isolation). Suppose the OS maps every process into the same physical address space with no MMU enforcement. A buffer overflow in the JPEG parser overwrites the banking app’s memory and steals the session cookie. With per-process virtual address spaces and page-table permissions enforced by the MMU, the same overflow can at most crash its own process—the fault is contained.

Isolation thus turns a total-system compromise into a compartment-limited failure, which defence-in-depth can then detect and recover from without trusting the faulty component.

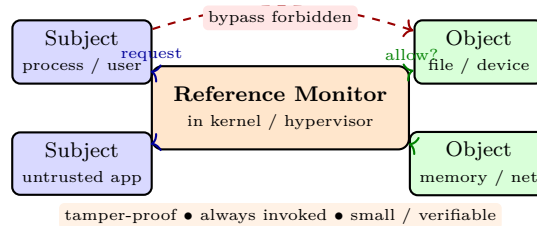


Figure 7.1: The reference monitor: every subject-to-object access is forced through a single, tamper-proof mediator. Dashed bypass must not exist.

7.2 The Reference Monitor and Complete Mediation

Anderson (1972) defined the *reference monitor* as an abstract machine that mediates all accesses. Three properties are non-negotiable:

Definition 7.2 (Reference monitor properties). A reference monitor must be (i) *always invoked* (complete mediation), (ii) *tamper-proof* (itself protected from modification), and (iii) *small enough to be verifiable* (amenable to analysis and testing).

In practice the kernel’s system-call path, the MMU/page tables, and the file-permission checks together implement the monitor. Hardware helps: CPU rings or privilege levels trap user-mode accesses; the kernel runs at higher privilege. Extended page tables and IOMMUs extend mediation to DMA from devices.

Example 7.2 (Bypassing mediation). A classic flaw: the OS checks `open("/safe/file")` but the actual read uses a file descriptor that an attacker swapped via a symlink (TOCTOU). The check and the use did not go through the same mediated path—property (i) violated. Fix: check and open atomically, or operate on file descriptors already obtained via the mediator.

Verification matters: seL4 is a microkernel with a machine-checked proof that its access-control enforcement satisfies the reference-monitor properties; larger monolithic kernels trade verifiability for performance and rely on code review, fuzzing, and privilege minimisation instead.

Remark 7.1 (Reference monitor vs. policy). The monitor is the *mechanism* that enforces; the *policy* (Bell–LaPadula for confidentiality, Biba for integrity, RBAC/ABAC rules) is what it enforces. Confusing the two leads to monitors that correctly enforce the wrong policy.

7.3 Privilege Separation

Privilege separation splits a program into compartments with different privilege levels so that a compromise of the less-trusted part does not yield full privileges. Think of a bank: the teller handles cash behind glass, but only the vault manager holds the vault key, and only for seconds.

Definition 7.3 (Privilege bracketing). Raise privilege, perform the minimal privileged operation, then drop privilege again. Never hold ambient authority while parsing untrusted input.

```

1 // privilege bracketing (pseudo-C, POSIX)
2 uid_t real = getuid(), eff = geteuid();
3 seteuid(real); // drop to unprivileged
4 parse_untrusted_input(buf); // bug here -> attacker gains only real uid
5 seteuid(eff); // re-acquire briefly
6 int fd = open("/var/log/auth", O_APPEND);
7 seteuid(real); // drop again immediately
8 write(fd, entry, len);

```

Other incarnations: OpenSSH splits `sshd` into a privileged monitor and an unprivileged slave communicating over a narrow IPC channel—even a full compromise of the pre-auth slave does not yield root. Chrome renders each tab in a low-privilege process; the browser process (broker) handles file and network access on its behalf. Linux capabilities refine this further by breaking root into fine-grained units (`CAP_NET_BIND`, `CAP_SYS_ADMIN`, `CAP_DAC_OVERRIDE`) so a process need not be fully root. Dropping capabilities with `cap_set_proc` and retaining only the needed one is least privilege in action.

Example 7.3 (setuid considered harmful). A file owned by root with the setuid bit executes with the owner’s privilege regardless of who launches it. A bug in a setuid helper (`/usr/bin/passwd`, ping) yields immediate privilege escalation. Modern systems minimise setuid binaries and prefer capabilities or a dedicated privileged daemon that validates requests.

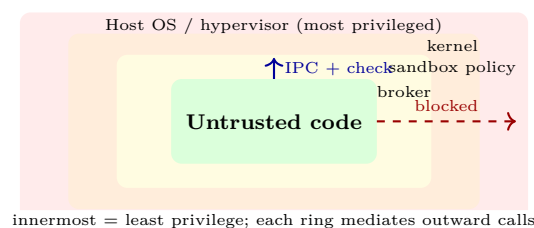


Figure 7.2: Layered privilege separation: untrusted code runs innermost; every sensitive operation must pass outward through broker and policy.

7.4 Sandboxing

A *sandbox* is a tightly confined execution environment that restricts what the code inside can do even if it is malicious or buggy. Kernel-enforced sandboxes and user-mode sandboxes complement each other.

System-call filtering. Linux `seccomp-bpf` lets a process install a BPF filter: only whitelisted syscalls proceed; others are killed or return an error. Firefox and Chrome use it to deny `execve`, `ptrace`, raw sockets to their renderers. OpenBSD `pledge` and `unveil` let a program voluntarily promise “I will only need `stdio` and `rpath`”; the kernel kills it if it breaks the promise. The call `pledge("stdio rpath", NULL)` is a one-liner that enforces least privilege with almost no code.

Capability and policy sandboxes. Capsicum (FreeBSD), Android app sandboxes (each app is a distinct UID with SELinux policy), and iOS entitlements restrict by capability tokens rather than ambient identity. Language-based sandboxes (WebAssembly, Java `SecurityManager` historically, browser JS) confine at the language runtime and complement OS sandboxes for defence-in-depth.

```

1 # seccomp-style allowlist (conceptual Python)
2 ALLOWED = {"read", "write", "exit", "sigreturn"}
3 def syscall_filter(name, args):
4     if name not in ALLOWED:
5         raise SandboxViolation(f"syscall {name} denied by policy")
6     return do_syscall(name, args) # mediated path
7 # After installing filter, even compromised parser cannot execve("/bin/sh")

```

Two design rules matter. First, sandbox *before* parsing untrusted bytes, not after—otherwise the attacker already ran code with ambient privilege. Second, keep the broker (the privileged process that serves sandbox requests) as small as possible; it is the new reference monitor and its bugs become sandbox escapes. Chrome’s broker validates every `open` request against a policy table before forwarding it to the OS.

7.5 Virtualization and Container Security

Virtualization runs multiple guest OSes on one host via a *hypervisor*. Type 1 (bare-metal: Xen, Hyper-V, KVM as module) runs directly on hardware; Type 2 (hosted: VirtualBox, VMware Workstation) runs atop a host OS. Hardware extensions (Intel VT-x, AMD-V, EPT/NPT for nested paging, IOMMU/VT-d for device DMA) make traps and memory virtualisation efficient. *Containers* (Docker, LXC, Kubernetes pods) share the host kernel but isolate via OS mechanisms: *namespaces* (PID, net, mount, UTS, user, IPC give each container its own view) and *cgroups* (limit CPU, memory, I/O), plus *seccomp*, AppArmor/SELinux, and dropped capabilities. Threats differ: a *VM escape* exploits a hypervisor bug (e.g. VENOM/QEMU floppy, CVE-2015-3456) to break into the host; a compromised guest that controls DMA without IOMMU can also attack host memory. Container escapes exploit kernel bugs or misconfiguration—privileged container, mounted Docker socket, `-privileged`, host-PID namespace, kernel CVE such as Dirty COW. Because containers share the kernel, a single kernel vulnerability breaks all containers on that host. Hence hardening is essential: run containers rootless, with read-only filesystems (`-read-only`), no `CAP_SYS_ADMIN`, user namespaces remapping container root to an unprivileged host UID, and *seccomp*/AppArmor profiles that deny dangerous syscalls. For

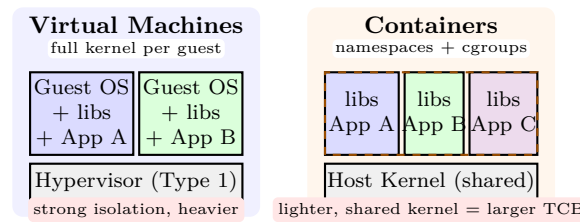


Figure 7.3: VMs vs. containers: VMs virtualise hardware with a hypervisor and guest kernels; containers share one kernel and isolate with namespaces/cgroups.

stronger isolation, gVisor inserts a user-space kernel (Sentry) that interposes syscalls, and Kata Containers wraps each container in a lightweight VM—trading some density for a hardware-enforced boundary. Choose VMs when tenants are mutually untrusted; choose containers when density and startup time dominate and the kernel TCB is acceptable.

7.6 Putting It Together: Hardening Checklist

A checklist ties the chapter together. Before shipping a service that handles untrusted input:

- **Enumerate trust boundaries.** What crosses from untrusted to trusted (file parses, IPC, device DMA)? Each crossing needs mediation.
- **Apply least privilege end-to-end.** Drop capabilities, use user namespaces, run renderers/workers as dedicated UIDs with no shell.
- **Sandbox before parse.** Install seccomp/pledge *before* the first byte of untrusted data is touched; verify the filter with `strace -c`.
- **Minimise broker TCB.** The broker that serves sandbox requests should validate, canonicalise, and log—no parsing of complex formats inside it.
- **Choose the right boundary.** Multi-tenant untrusted code → VMs or Kata/gVisor; single-tenant microservices → containers with tight seccomp/AppArmor.
- **Plan for escape detection.** Monitor for anomalous syscalls, unexpected capability use, and hypervisor/container CVEs; have a patch and live-migration story.

Example 7.4 (Hardening a document converter). A service converts uploaded PDFs to images. Bad design: a monolithic root process calls `libpoppler` on the upload. Good design: an unprivileged worker in a seccomp + mount-namespaces sandbox with no network, read-only rootfs, and a tiny broker that only allows `open` of the single input file and `write` of the output. Even a full RCE in `libpoppler` yields only a sandboxed UID that cannot exec or exfiltrate.

Remark 7.2 (Common pitfalls). Overly broad seccomp allowlists (allowing `ptrace` or `mount`), writable `/proc` or Docker socket mounts, and retaining `CAP_SYS_ADMIN` “just in case” are the three most common sources of container escapes seen in CTFs and incident reports.

Example 7.5 (Why small TCB matters). The classic qmail SMTP server (Bernstein) was split into mutually untrusting modules so that the parser that speaks to the network never runs as root. Despite decades of exposure, no remote root hole has been found—splitting and minimising the privileged part paid off. The same

lesson drives seL4, gVisor's Sentry, and Chrome's broker: bugs are inevitable, but their location determines impact.

Theorem 7.1 (Isolation reduces blast radius). If compartment C_1 is compromised but C_1 holds no capability for object O in C_2 and every path to O is mediated, then C_1 cannot directly violate C_2 's integrity or confidentiality except via covert or side channels outside the modeled mediator.

In practice side channels (Spectre, Meltdown, cache timing) complicate the picture, which is why hardware isolation (page tables, EPT, IOMMU) must be combined with software hardening and constant-time handling of secrets.

7.7 Exercises

- E7.1 **Reference monitor audit.** Your OS checks permissions on `open()` but a background thread can replace the pathname with a symlink between check and use (TOCTOU). Which reference-monitor property fails? Propose a fix using file descriptors or `openat2(RESOLVE_NO_SYMLINKS)` and argue why it restores the property.
- E7.2 **Privilege bracketing.** A server parses a JPEG while holding `CAP_SYS_ADMIN`, then writes the log. Sketch an exploit if the parser has a buffer overflow. Rewrite the flow with bracketing/passing to a helper and explain why the blast radius shrinks.
- E7.3 **seccomp policy design.** A PDF renderer needs `read`, `write`, `mmap`, `exit` but not `execve` or `socket`. Write a seccomp-bpf allowlist and describe what happens if the renderer is compromised and tries `execve("/bin/sh", ...)`. What must the broker do for legitimate file-open requests?
- E7.4 **VM vs. container trade-off.** You must run 100 mutually untrusted customer workloads on one host. Compare (a) one VM per customer vs. (b) one container per customer on (i) isolation strength, (ii) memory/CPU overhead, (iii) patching burden. When would you choose Kata Containers/gVisor and why?
- E7.5 **Container escape via misconfiguration.** A container is started with `docker run -privileged -v /:/host ubuntu bash`. Explain step-by-step how an attacker inside the container obtains host root. List three hardening bugs that would block this path.

Chapter Notes

Reference monitor and Anderson report (1972); Bell–LaPadula/Biba as policy models for the monitor. Least privilege (Saltzer & Schroeder 1975). Privilege separation in OpenSSH (Provos et al. 2003), Chrome site isolation and sandbox (Reis & Gribble). seccomp-bpf (Drewry 2012), OpenBSD `pledge/unveil`, Capsicum, seL4 verified kernel (Klein et al. 2009). Virtualization: Popek & Goldberg (1974) requirements, Intel VT-x/AMD-V, EPT/NPT, IOMMU/VT-d. Containers: namespaces/cgroups, Docker, Kubernetes; hardening via rootless, gVisor (user-space kernel), Kata Containers (lightweight VMs). For a deeper view of capability systems see Capsicum (Watson et al.) and Fuchsia's capability-based security model.

Chapter 8

Applications: Privacy, Forensics, Incident Response, and Law

“Security is not only cryptography and firewalls; it is what happens when things fail, how we prove what happened, and what the law requires us to do.” — This chapter connects technical mechanisms to privacy guarantees, evidence, operations, and regulation.

From zero: we build here, no law or statistics course needed. We assume only that you have collected or seen data tables (rows = people, columns = attributes) and that organisations must respond when systems break. From those everyday starting points we formalise privacy, forensics, response, and GDPR obligations—pictures before definitions.

Learning Outcomes

After this chapter you will be able to:

- (L1) define k -anonymity, its limitations, and l -diversity/ t -closeness refinements;
- (L2) state ϵ -differential privacy and apply the Laplace mechanism;
- (L3) explain chain-of-custody and forensically sound handling of digital evidence;
- (L4) describe the NIST incident-response lifecycle and map actions to each phase;
- (L5) summarise GDPR principles, lawful bases, data-subject rights, and breach-notification duties.

8.1 Privacy: From Removing Names to Formal Guarantees

Removing names is not anonymisation. Latanya Sweeney showed 87% of Americans can be re-identified by {ZIP, birthdate, gender} alone—*quasi-identifiers* combine with public voter rolls to single out a row. AOL search-log and Netflix Prize de-anonymisations repeat the lesson: linkage attacks defeat naive scrubbing. We

thus distinguish: *identifier* (name, NRIC, email—directly identifying), *quasi-identifier* (ZIP, age, gender—identifying in combination), and *sensitive attribute* (disease, salary) we wish to protect.

Raw table			2-anonymous table		
ZIP	Age	Disease	ZIP	Age	Disease
62038	34	Cancer	6203*	30–39	Cancer
62039	34	Flu	6203*	30–39	Flu
62012	45	Diabetes	6201*	40–49	Diabetes
62013	45	Cancer	6201*	40–49	Cancer

equiv. class size 2
 every quasi-id group has $\geq k$ rows

Figure 8.1: Generalisation to achieve k -anonymity ($k = 2$): ZIP truncated and ages bucketed so each quasi-identifier tuple appears at least k times. Suppression (*) trades utility for indistinguishability.

Definition 8.1 (k -anonymity). A table satisfies k -anonymity if every combination of quasi-identifier values appears in at least k rows. Each such group is an *equivalence class*.

Mechanisms: *generalisation* (replace 62038 with 6203*, 34 with 30–39) and *suppression* (delete outlier rows). Trade-off: larger k means stronger indistinguishability but coarser data.

Example 8.1 (Homogeneity attack). A 2-anonymous class $\{(6203*, 30-39): \text{Cancer}, \text{Cancer}\}$ is 2-anonymous yet reveals the sensitive value—everyone in the class has Cancer. k -anonymity does not protect attribute disclosure.

Refinements: *l-diversity* requires each class to contain at least l well-represented sensitive values; *t-closeness* requires the distribution of the sensitive attribute in each class to be within distance t of the global distribution. None is composable or robust to background knowledge—hence the move to differential privacy.

8.2 Differential Privacy

Differential privacy (DP) gives a mathematical promise about an *algorithm*, not a static table: its output distribution barely changes if one person’s row changes. The adversary may know all other rows.

Definition 8.2 (ϵ -DP). A randomised mechanism $\mathcal{M}$ is ϵ -differentially private if for all neighbouring datasets D, D' differing in one row and all output sets S ,

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S].$$

Smaller ϵ means stronger privacy; typically $\epsilon \in [0.1, 5]$.

Intuition: the factor $e^\epsilon \approx 1 + \epsilon$ for small ϵ bounds how much one person can influence the output—plausible deniability. DP composes: k ϵ_i -DP releases together are $(\sum \epsilon_i)$ -DP (basic composition); privacy budget must be managed.

Laplace mechanism. For a numeric query f with *global sensitivity* $\Delta f = \max_{D \sim D'} |f(D) - f(D')|$, adding Laplace noise scaled to $\Delta f/\epsilon$ yields ϵ -DP.

$$\mathcal{M}(D) = f(D) + Y, \quad Y \sim \text{Lap}(\Delta f/\epsilon), \quad p_Y(y) = \frac{\epsilon}{2\Delta f} e^{-\epsilon|y|/\Delta f}.$$

Counting queries have $\Delta f = 1$; summing salaries has sensitivity equal to max salary range.

```

1 import numpy as np
2 def laplace_mech(true_answer, sensitivity, epsilon):
3     scale = sensitivity / epsilon          # larger epsilon -> less noise
4     noise = np.random.laplace(0, scale)
5     return true_answer + noise
6 # Example: count of patients with disease; sensitivity=1, epsilon=0.5
7 noisy_count = laplace_mech(42, sensitivity=1, epsilon=0.5)

```

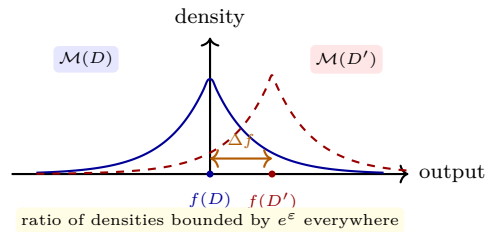


Figure 8.2: Differential privacy: neighbouring datasets induce nearby output distributions (Laplace here). No output is much more likely under D than under D' .

Local vs. central DP, (ϵ, δ) -DP relaxations, and composition accounting (moments accountant) extend the model; randomised response is the local-DP analogue for surveys.

8.3 Digital Forensics and Chain-of-Custody

When a breach becomes a legal matter, *digital forensics* must answer what happened in a way a court can trust. The process is: *identify* $\rightarrow$ *preserve* $\rightarrow$ *collect* $\rightarrow$ *examine* $\rightarrow$ *analyse* $\rightarrow$ *report*, with integrity at every step.

Forensically sound handling. Work on a *bit-for-bit image*, never the original; attach a write blocker; hash immediately (SHA-256) and re-hash at every transfer to prove the image has not changed. Document tool versions and commands so another analyst can reproduce the result.

Chain-of-custody. A chronological, signed record of who handled the evidence, when, why, and what was done. Gaps or undocumented access render evidence inadmissible—even a perfect forensic image is worthless if you cannot prove it is the image of the seized disk. Store hashes and custody forms separately from the evidence; timestamp with a trusted source.

Order of volatility (RFC 3227). Collect most ephemeral first: registers/cache $\rightarrow$ RAM $\rightarrow$ network state $\rightarrow$ running processes $\rightarrow$ disk $\rightarrow$ backups/archives. Pulling the power preserves disk but destroys RAM artefacts (in-memory malware, decryption keys); live acquisition may be required but must be justified and logged.

```

1 # forensically sound imaging (conceptual)
2 import hashlib
3 def image_and_hash(src_path, dst_path):
4     h = hashlib.sha256()
5     with open(src_path, 'rb') as src, open(dst_path, 'wb') as dst:
6         while chunk := src.read(1<<20): # 1 MiB at a time

```

```

7         h.update(chunk); dst.write(chunk)
8     digest = h.hexdigest()
9     log(f"SHA-256({dst_path})={digest}  tool=v1.2  examiner=Alice  time=2026-08-28T09:00Z"
10        )
11     return digest
12 # Re-hash before analysis; mismatch -> custody failure

```

Anti-forensics (timestamping, log wiping, steganography) must be anticipated; corroborate across independent sources (disk, netflow, cloud audit logs).

8.4 Incident Response: The NIST Lifecycle

Incident response (IR) is the operational discipline of handling breaches. NIST SP 800-61 Rev. 2 organises it into four phases forming a cycle:

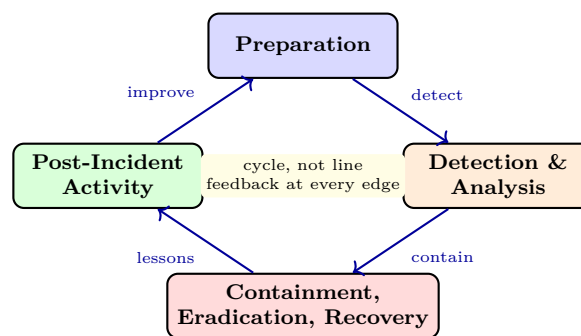


Figure 8.3: NIST incident-response lifecycle: preparation enables every other phase; post-incident feeds back into preparation.

1. Preparation. Policies, playbooks, asset inventories, logging/monitoring, IR team roles (technical, legal, comms), tabletop exercises. Without preparation the other phases improvise.

2. Detection & Analysis. Triage alerts (IDS, EDR, SIEM), scope the incident, preserve evidence forensically, assign severity. Key question: is this an *event* or an *incident* (adverse impact)?

3. Containment, Eradication & Recovery. *Short-term* containment (isolate host, block IP) stops bleeding; *long-term* containment hardens. Eradicate root cause (remove backdoor, patch, rotate creds). Recover by restoring from known-good backups and monitoring for resurgence. Every action is logged for custody and later review.

4. Post-Incident Activity. Lessons-learned meeting, root-cause analysis, metrics (MTTD/MTTR), update playbooks and controls. The output is a report that improves Preparation—closing the loop. Prioritisation uses impact vs. recoverability; communication plans separate internal, customer, regulator, and public messaging to avoid contradictory disclosures.

8.5 Regulation: GDPR Essentials

The EU General Data Protection Regulation (2016/679) applies to any organisation processing personal data of EU residents, regardless of where the organisation sits. Key ideas:

Principles (Art. 5). Lawfulness, fairness, transparency; purpose limitation; data minimisation; accuracy; storage limitation; integrity/confidentiality; accountability (you must *demonstrate* compliance).

Lawful bases (Art. 6). Processing needs one of six bases: consent, contract, legal obligation, vital interests, public task, or legitimate interests (balanced against rights). Consent must be freely given, specific, informed, unambiguous, and withdrawable.

Roles. *Data subject* (person), *data controller* (decides purposes/means), *data processor* (acts for controller), *DPO* and *supervisory authority*.

Rights of the data subject. Access, rectification, erasure (“right to be forgotten”), restriction, portability, objection, and not to be subject to solely automated decision-making (Art. 12–22).

Obligations & breach notification. Data Protection Impact Assessments for high-risk processing, records of processing, security by design/default (Art. 25, 32). Breach notification to the authority within **72 hours** of awareness (Art. 33) and to affected subjects without undue delay if high risk (Art. 34). Fines up to €20m or 4% of global annual turnover, whichever is higher. Other regimes (Singapore PDPA, US sectoral laws, CCPA/CPRA) differ in scope and enforcement but share the trajectory: minimise data, justify retention, notify promptly, and be auditable.

8.6 Budget, Custody, and Notification: A Worked Scenario

A mid-size SaaS company runs a containerised analytics platform. Three vignettes show how the chapters connect.

Vignette A: privacy budget. The product team wants to publish daily counts of active users by region and by plan tier over a month (60 queries). Each is $\varepsilon = 0.2$ -DP via Laplace. Basic composition gives total $\varepsilon = 12$ —too large (almost no guarantee). Using advanced composition or splitting ε to 0.05 per query and adding a little more noise, or publishing only weekly aggregates, brings the monthly budget back under $\varepsilon = 3$. The lesson: privacy, like incident response, needs budgeting before release, not after.

Vignette B: forensics meets IR. An attacker exploits a container escape (mounted Docker socket, as in E7.5) and wipes `/var/log/auth.log`. Because the IR playbook mandated centralised, append-only log shipping and hourly disk images with SHA-256 chain-of-custody, the team reconstructs the attack from the SIEM and the previous image. Without custody, the wiped log would have been taken as ground truth.

Vignette C: GDPR clock. Forensics confirms the attacker exfiltrated a user table that included EU residents. The 72-hour clock under Art. 33 started when the controller had “reasonable degree of certainty” (forensic confirmation), not when the IDS first fired. The controller notifies the supervisory authority within the window, documents the delay rationale, and notifies high-risk data subjects after scoping—exactly the post-incident reporting envisaged by NIST. Together the three vignettes illustrate a single principle: technical guarantees (DP, isolation), operational discipline (forensics, IR), and legal duties (GDPR) must be designed jointly, because a failure in one undermines the others.

Theorem 8.1 (Composition and utility trade-off). Let $\mathcal{M}_1, \dots, \mathcal{M}_k$ each be ε_i -DP. Then the sequential release $(\mathcal{M}_1, \dots, \mathcal{M}_k)$ is $(\sum_i \varepsilon_i)$ -DP. Reducing each ε_i improves privacy but increases noise variance $\propto (\Delta f / \varepsilon_i)^2$, degrading utility. Choosing ε is thus a policy decision balancing re-identification risk against data usefulness, and must be audited alongside GDPR data-minimisation.

Example 8.2 (GDPR meets DP). A hospital wants to publish average length-of-stay by diagnosis. Instead of releasing k -anonymous microdata (vulnerable to linkage via rare diagnoses), it releases Laplace-noised averages with $\varepsilon = 1.0$ and documents the privacy budget in its Record of Processing Activities (RoPA). This satisfies purpose limitation and minimisation while giving a formal guarantee the supervisory authority can audit—a concrete bridge from mathematics to compliance.

Remark 8.1 (Forensics readiness). Logging with hash-chained integrity (e.g., `syslog` + RFC 3161 timestamps) and hardware-rooted attestation (TPM-measured boot) turns post-incident reconstruction from guesswork into checkable evidence. Readiness is cheaper than retroactive collection and is explicitly encouraged by GDPR Art. 32 (integrity and resilience).

Example 8.3 (Checklist: 72-hour clock). Hour 0: IDS alert $\rightarrow$ SOC creates ticket, preserves RAM/disk images with hashes. Hour 4: Forensic analysis confirms personal data exfiltration $\rightarrow$ awareness for Art. 33. Hour 12: DPO drafts notification (nature, categories, approximate subjects, consequences, mitigation). Hour 48: Authority notified; if risk is high, affected subjects notified directly. Hour 72: Deadline for authority notification; document reasoning if any delay. Every step is logged in the chain-of-custody register so the supervisory authority can audit timeliness and completeness.

Remark 8.2 (Singapore contrast). Singapore’s PDPA also requires breach notification (since 2021) but with a 3-calendar-day window to the PDPC and notification to affected individuals if significant harm is likely. Comparing timelines shows why multinationals map a single playbook to the strictest deadline—GDPR’s 72 hours—and add local annexes.

8.7 Exercises

E8.1 k -anonymity construction. Table has quasi-identifiers {ZIP, Age} and sensitive Disease. Generalise ZIP to 3 digits and bucket ages into 10-year intervals to achieve $k = 3$. Show the resulting equivalence classes. Give a homogeneity attack on your table and explain how l -diversity would block it.

E8.2 Differential privacy calculation. A counting query has sensitivity $\Delta f = 1$ and mechanism $M(D) = f(D) + \text{Lap}(1/\varepsilon)$ with $\varepsilon = 0.5$. Compute $\Pr[|M(D) - f(D)| > 2]$ and the factor e^ε bounding neighbouring likelihood ratios. If you answer 10 such queries with basic composition, what is the total ε ?

- E8.3 **Chain-of-custody failure.** An analyst images a disk, forgets to hash it, edits the image to extract a file, then hashes. Opposing counsel moves to exclude the evidence. Explain why the chain is broken, what the correct forensically sound workflow is, and how order of volatility should have guided collection if RAM contained the decryption key.
- E8.4 **NIST lifecycle tabletop.** A ransomware alert fires at 02:00 on a file server. Map the next 48 hours to the four NIST phases, listing concrete tasks, owners (SOC, IT, legal, comms), and evidence-preservation steps. Where does the 72-hour GDPR clock fit?
- E8.5 **GDPR analysis.** A Singapore e-commerce site tracks EU visitors' browsing via cookies for ads (a) without consent and (b) with a pre-ticked box. Identify the lawful basis (if any), which GDPR principles are violated, which data-subject rights are engaged, and the breach-notification duty if the cookie database leaks.

Chapter Notes

Privacy: Sweeney (2002) k -anonymity; Machanavajjhala et al. (2007) l -diversity; Li et al. (2007) t -closeness; Dwork et al. (2006) differential privacy, Laplace mechanism; Dwork & Roth (2014) survey. Forensics: Carrier (2005), RFC 3227 volatility, NIST SP 800-86. Incident response: NIST SP 800-61 Rev. 2, MITRE ATT&CK for detection mapping. GDPR: Regulation 2016/679; see Art. 5, 6, 12–22, 25, 32–34; Singapore PDPA (2012, rev. 2020) for contrast; EDPB guidelines on consent and breach notification.